

Advance Java

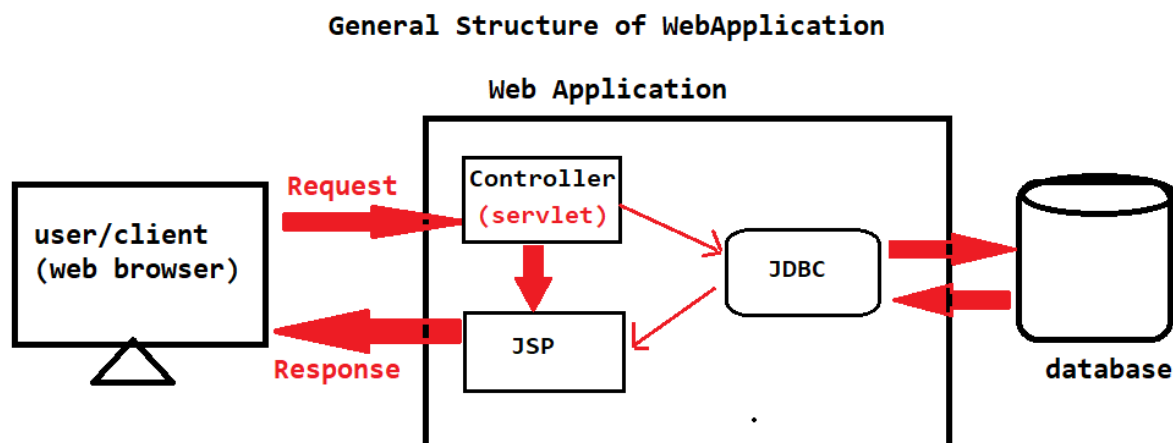
->In advance java we will learn about 3 technologies

- JDBC (java database connectivity)
- Servlet
- JSP

->using advance tech stack we can develop web application.

What is web application ?

->In simple word the application which is executing in the internet environment or web environment is known as WebApplication. In which request and response play a major role.



Here

JDBC:-stands for **java database connectivity** which helps to interact with database.

Servlet:-Servlet is a server side program which accept the request from client/user through web browser.

JSP:-JSP stands for **Java server page** and which is response from web application to client.

JDBC

->Before understand about JDBC let understand about storage first.

Types of Storages related to JAVA

- Field Storage
- Object Storage
- File Storage
- Database Storage

Field Storage:-Field Storage are related to our variable and this memory is generated to hold single data value and this is also a temporary storage in program. All primitive datatype variables are nothing but field storage only.

Object Storage:-The memory generated to hold group of members is known as Object Storage. This is generated in heap memory and it is also a temporary storage.

Note:- both Field Storage and Object Storage are temporary storage because these storage will be destroyed automatically when JVM shutdown.

When we have to store something permanent then we have two types of storage .

- File Storage
- Database Storage

File Storage:- It is a Storage which is managed by operating System and it is recommended for File storage Not for Web application Data ,

->File Storage is ok for local files and data but if we talk about any web application perspective then there is lot of limitation due to which we are unable to use File Storage for web application.

Few Limitation/disadvantage of File Storage

Data Redundancy:-

->redundancy means duplication of data ,in file system it is possible to have duplicate data which leads to memory wastage.

Data Inconsistency:-

->Data Inconsistency means data are not present in well organized manner as well as not proper consistent state.

Limited Data Sharing:-

->all file and data may present in scattered form unorganized manner which create problem to share data properly.

Atomicity Problem:

->Any operation on database must be atomic, which means it must happen entirely or not at all. It is possible in database but in file storage atomicity is impossible.

Multiple-access

->In database multiple users are allowed to access data at same time simultaneously means they can perform reading /writing operation same time which is not possible in File Storage.

Security

->as compared to File Storage Security is very high in database .

Database Storage

->database is the permanent storage which is installed into computer.which is used to store data of web application (client details,application details..etc).

->There are lots of database available in market like Mysql,Oracle..etc.

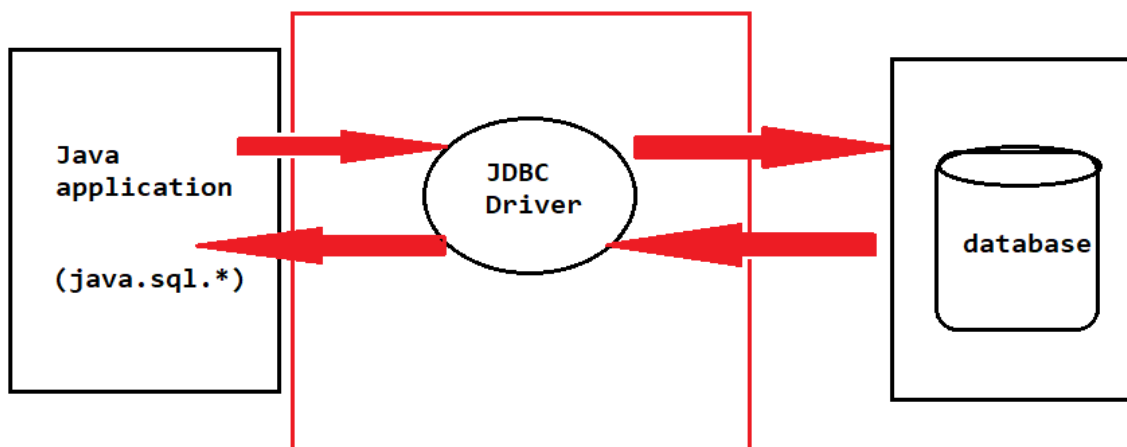
->In database we can store data in well organized format which make easy to insert and fetch data from it.

->it solves all the data storage issue present in File Storage.

->lets talk about JAVA application.

Developer can use any database its depends on there requirement. So when we want to establish communication between database like(Mysql,Oracle..etc) then java app must be developed with such Classes and Interfaces that which helps to establish communication between java application and database.

->java need to take help of "java.sql" package and need support of JDBC driver.



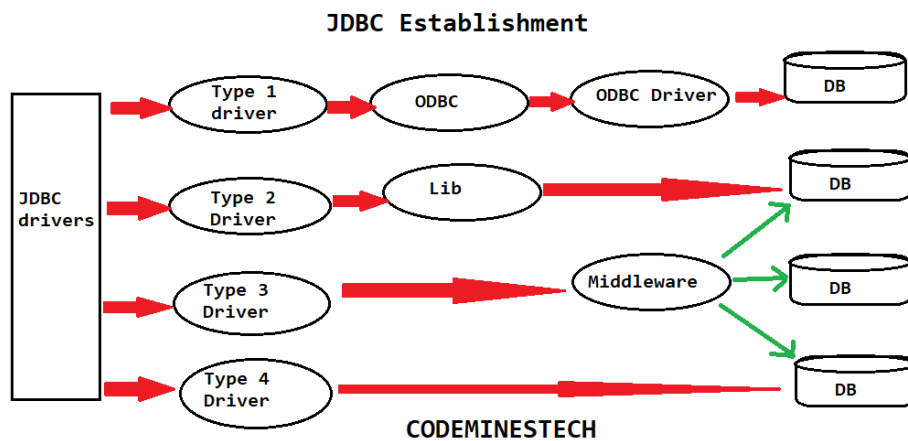
Driver:-

->JDBC driver is a software it enable java application to interact with database.

->Driver is acting as a translator between java application and database . and it converts java style SQL queries into Database supported style SQL queries.

JDBC drivers are categorized into following.

- JDBC-ODBC bridge driver(type -1 driver)
- Native API driver (type-2 driver)
- Network Protocol driver (Type -3 driver)
- Thin driver (Type-4 driver)



JDBC-ODBC bridge driver(Type-1):

-> In Type-1 driver, JDBC Call is converted into ODBC Call and this ODBC call is converted into Database Specific call for Connection.

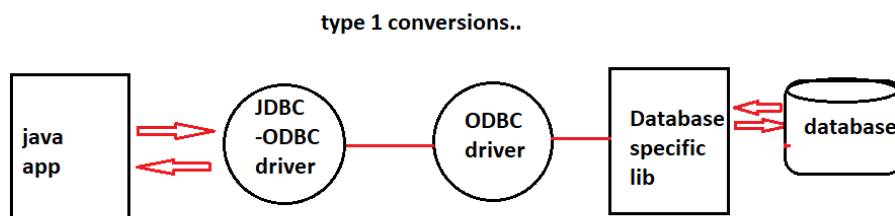
DisAdvantage:

->Type-1 driver having more number of conversions and consumes more execution time and degrades the performance.

-> we need to install Microsoft provided ODBC native library in our Machine.

->more conversion happens like **Java to ODBC** and from **ODBC to database** native.

->Type 1 driver is discontinued from java 8 version.



->ODBC Stands for 'Open DataBase Connectivity' and which is used to Convert Java Calls(JDBC calls) into Native ODBC calls.

->ODBC is PlatForm dependent driver,becuase which internally having c/c++ code.

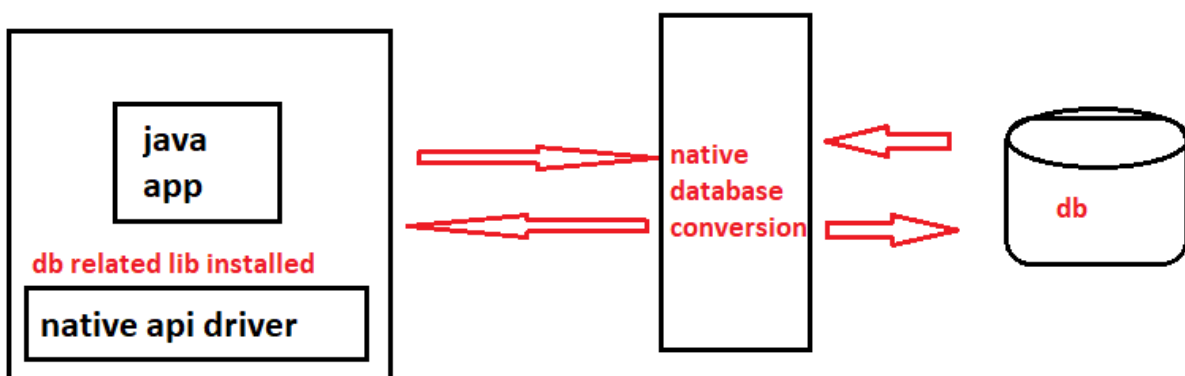
Native API driver(Type-2)

-> In Type-2 driver,the Client Machine is installed with DataBase related Libraries these Libraries will Support Type-2 driver to convert JDBC calls into DataBase Specific calls for connection.

DisAdvantage:

->In Type-2 driver the Application will become DataBase dependent application.

Type 2



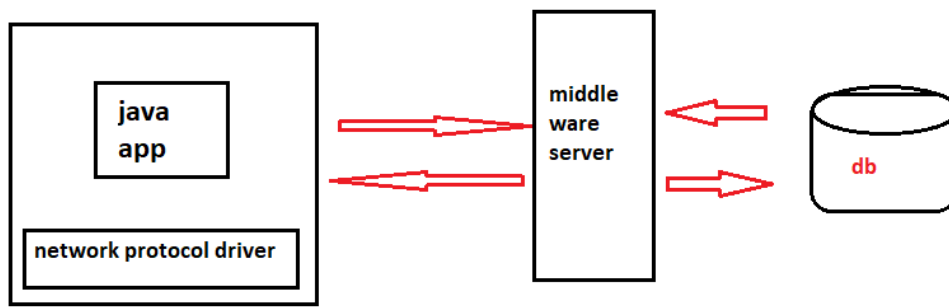
Network Protocol driver(Type-3):

-> In Type-3 driver,we make a call to MiddleWare-Server and this MiddleWare Server will establish connection to DB product.

DisAdvantage

-> In Type-3 driver,more number of N/W components are participated in execution process and increases the execution time and degrades the performance of an application.

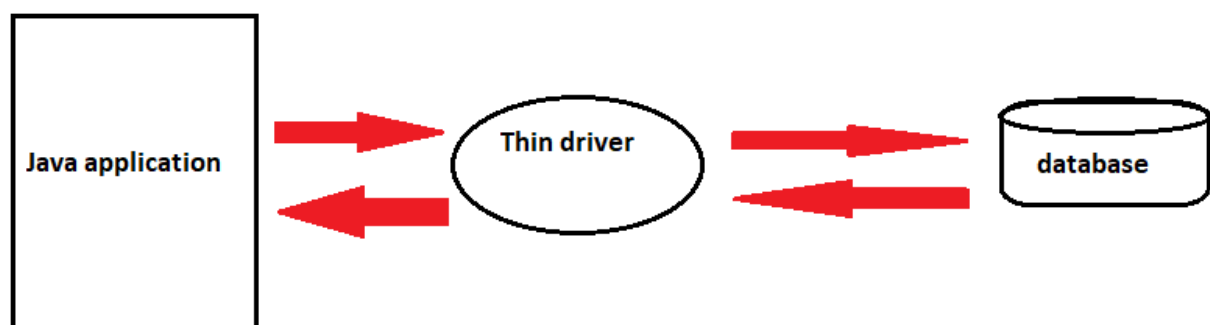
Type 3



Thin driver(Type-4): (imp)

- >This Type -4 driver is also called as **Thin Driver** or **Pure Java** Driver
- >This is pure Java based driver
- >It is most portable as compared to other 3 drivers Types.
- >This type-4 driver not requires any middleware and all.
- >We can use Type-4 driver for any kind of Java application development.
- >it is Fast.
- >here JDBC calls are converted into database specific calls directly.
- >high performance driver.

TYPE 4



->now we have already learn Oracle so in advance java we will use Oracle and in spring boot we use Mysql.

->if you have not downloaded Oracle database software then first download it.

->as you download Database we need its database jar file support to connect with that database.

Every database comes with its jar file like Oracle

Oracle:-ojdbc.jar (version may different)

Mysql:-mysql-connector-java.jar (version may diff)

Postgresql:- postgresql.jar

Etc.

These Jar files is compressed format of Classes related to database.

Note:-*DB Jar file is available at lib of Oracle product:*

C:\oracle\app\oracle\product\11.2.0\server\jdbc\lib

(in you case location may be different .. if unable to find then you can directly download from internet also).

->Database Port and Service Name

->every database have its own port no and serviceName.

To know port no and service name of oracle open 'tnsnames.ora' file present inside oracle folder .

Ex:- **C:\oracle\app\oracle\product\11.2.0\server\network\ADMIN** (location depend on which drive your oracle present)

This PC > New Volume (F:) > oracle > dbhomeXE > network > admin

Name	Date modified	Type	Size
sample	17/08/2021 5:40 PM	File folder	
listener.ora	17/08/2021 5:45 PM	ORA File	1 KB
sqlnet.ora	17/08/2021 5:45 PM	ORA File	1 KB
sqlnet2108175PM4506.bak	17/08/2021 5:45 PM	BAK File	1 KB
tnsnames.ora	17/08/2021 6:10 PM	ORA File	1 KB

tnsnames.ora Network Configuration File: F:\oracle\dbhomeXE\NETWORK\ADMIN\tnsnames.ora
Generated by Oracle configuration tools.

```
XE =  
(DESCRIPTION =  
  (ADDRESS = (PROTOCOL = TCP)(HOST = DESKTOP-OJAORIM)(PORT = 1521))  
  (CONNECT_DATA =  
    (SERVER = DEDICATED)  
    (SERVICE_NAME = XE)  
  )  
)  
  
LISTENER_XE =  
  (ADDRESS = (PROTOCOL = TCP)(HOST = DESKTOP-OJAORIM)(PORT = 1521))
```

DataBase API:

->'java.sql' is known as DataBase API or JDBC API, which provides related classes and Interfaces used to develop JDBC Application.

->'java.sql.Connection' is an interface and which is root of JDBC api.

These are some important method of Connection Interface

(a)createStatement()

(b)prepareStatement()

(c)prepareCall()

(d)setAutoCommit()

(e)getAutoCommit()

(f)commit()

(g)rollback()

(h)setSavepoint()

(i)releaseSavepoint()

(j)close()

->as we discuss '**Connection**' is an interface and we also know that we can't create object for Interface .

->we can use '**getConnection()**' method to create the implementation object of 'Connection interface'.

->This **getConnection()** method is available from '**java.sql.DriverManager**'.

Method Signature of getConnection() method:

```
public static java.sql.Connection getConnection(java.lang.String,  
java.lang.String,java.lang.String)throws java.sql.SQLException;
```

syntax: -

```
Connection con = DriverManager.getConnection(url,username,password);
```

Url=>**jdbc:oracle:thin:@localhost:1521:xe**

username=>**system**

password =>**manager** (in my case it's root)

ex:

```
Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","manager");
```

API ↓
driver ↓
port No ↓
jdbc:oracle:thin:@localhost:1521:xe
↑
Product location Service Name

Connection (interface)

these are some
non-static usefull
methods in
Connection .

```
createStatement()  
prepareStatement()  
prepareCall()
```

```
Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","manager");
```

ex:- con.createStatement();

ex:- con.prepareStatement();

ex:- con.prepareCall();

Types of Statements in JDBC:

=>Statements in JDBC are categorized into **three** types:

- **1.Statement**
- **2.PreparedStatement**
- **3.CallableStatement**

Above three statements contains various method which helps us to deal with database queries. Let see some imp methods.

Statement:

->'Statement' is an interface from java.sql package and which is used to execute normal queries like **create** , **insert** , **reterive** (select), **update** and **delete** by providing some method support.

These are some important methods from 'Statement' .

- `executeQuery()`
- `executeUpdate()`
- `execute()`

ex:

```
Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","manager");  
Statement stmt = con.createStatement();  
    stmt.executeQuery(paramter);  
    stmt.executeUpdate(paramter);
```

executeQuery():

->executeQuery() method is used to execute 'select' queries.

Method Signature:

```
public abstract ResultSet executeQuery(String) throws java.sql.SQLException;
```

executeUpdate()

-> executeUpdate() method is used to execute **Non-Select queries** like **Create, Insert, Update** and **Delete**.

Method Signature:

```
public abstract int executeUpdate(String) throws java.sql.SQLException;
```

Syntax:

```
Statement stm = con.createStatement();
```

```
ResultSet rs = stm.executeQuery("select-query");
```

```
int k = stm.executeUpdate("Non-Select-Query");
```

Execute()

-> execute() method can be used to execute any kind of sql statements 'select' as well as 'non-select'.

Method Signature:

```
public abstract boolean execute(parameter) throws java.sql.SQLException;
```

Difference between these 3 methods:

executeQuery()	executeUpdate()	execute()
This method is used to execute SQL statement which retrieve some data from database This method returns a ResultSet object which contains the result returned by the query. This method is used to execute only select queries. ex:- select	This method is used to execute the sql statement which update or modify the database. This method returns an int value which represents the number of rows affected by the query. returns 0 for the query which return nothing. This method is used to execute only non-select queries. ex:- insert, update, delete, create, alter..	This method can be used for any kind of SQL statement. this method return boolean value This method can be used for both select & non-select queries.

Constructing JDBC Application Using IDE Eclipse:

Step 1:- Open IDE eclipse .select workspace and launch it.

Step2:- create java project

Click on File → new → project → java → java project → click next → name the project → click finish.

Step3:- add Database –jar library to our java project in our case (ojdbc).

Right click on project → build path → configure build path → libraries → select classpath → click on ‘ add external jars’ → browser and select db jar → apply and apply close.

Step 4:- you are ready to construct program.

Task:- create a java program to fetch all records of employee table

Our db table

EMPID	ENAME	JOB	SAL	HIREDATE	AGE
100	imran	dev	70000	28/OCT/23	21
101	sumit	dev	75000	29/OCT/23	21
103	ganesh	tester	80000	27/NOV/24	21
104	akshay	dba	10000	01/JAN/24	22

forName() is not needed since JDBC 4.0. In previous versions of JDBC, to obtain a connection, you first had to initialize your JDBC driver by calling the method `Class.forName()`.

```

package test;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DatabaseTest {

    public static void main(String[] args) {

        try {
            Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");
            Statement stm = con.createStatement();
            ResultSet rs = stm.executeQuery("select * from employee");
            while (rs.next()) {
                System.out.println(
                    rs.getInt(1) + "\t" + rs.getString(2) +
                    "\t" + rs.getString(3) + "\t" + rs.getFloat(4) +
                    "\t" + rs.getDate(5) + "\t" + rs.getInt(6));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

url username password
 ↑ ↑ ↑
 executeQuery()
 return ResultSet now
 we need to iterate
 it to get data one
 by one.

ResultSet?

-> ResultSet is an interface from java.sql package and which holds the result of 'select' query.

-> executeQuery() method return the implementation object of ResultSet interface. That object will hold select query result.

Following important methods of ResultSet

(i) getString()

(ii) getInt()

(iii) getFloat()

(iv) getLong()

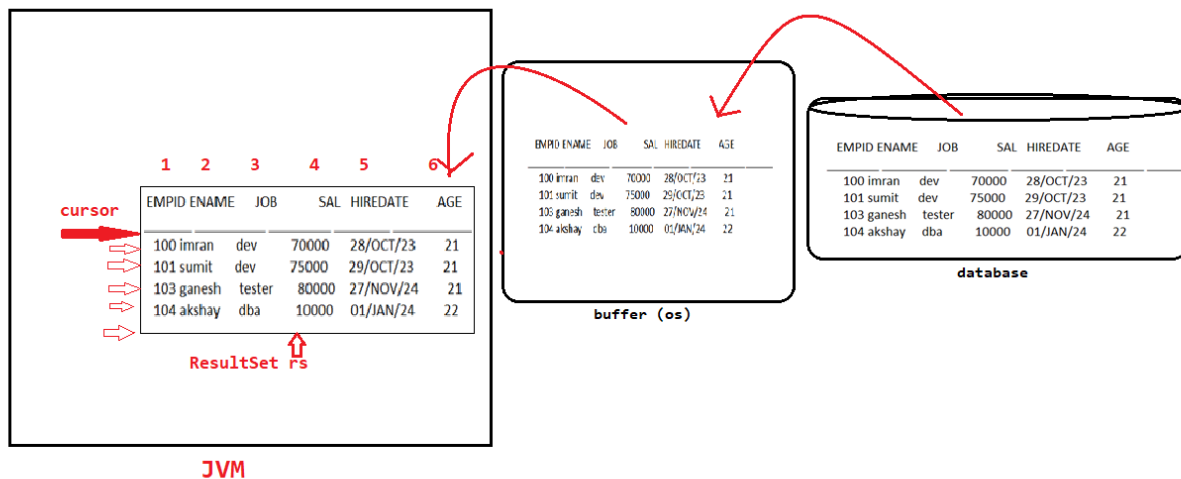
(v) next()

next() method:

=> next() method move the cursor row-by-row on ResultSet object

=> return type of next() method is 'boolean'

=> next() method returns 'true' when record available, else returns 'false'



Task: create java program to insert data in employee table

```
public class Databasetest2 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // EMPID ENAME JOB SAL HIREDATE AGE

        System.out.println("enter employee id");
        String empid = sc.nextLine();

        System.out.println("enter emp name ");
        String ename = sc.nextLine();
        System.out.println("enter job name");
        String jobName = sc.nextLine();
        System.out.println("enter date");
        String date = sc.next();
        System.out.println("enter salary");
        int sal = sc.nextInt();
        System.out.println("enter age");
        int age = sc.nextInt();

        try {
            Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");
            Statement stm = con.createStatement();

            try {
                Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");
                Statement stm = con.createStatement();

                // Parse the user input string into a Date object
                SimpleDateFormat inputDateFormat = new SimpleDateFormat("dd-MM-yy");
                Date userdate = inputDateFormat.parse(date);

                // Format the date in the way Oracle expects
                SimpleDateFormat oracleDateFormat = new SimpleDateFormat("dd-MMM-yy");
                String oracleFormattedDate = oracleDateFormat.format(userdate);

                String query = "insert into employee values('"+ empid + "','" + ename + "','" + jobName + "','" + sal +
                    "','TO_DATE('"+ oracleFormattedDate + "','"+ "age + " '))";

                long inserted = stm.executeUpdate(query);

                if (inserted > 0) {
                    System.out.println("employee data inserted succesfully...");
                } else {
                    System.out.println("something went wrong!!");
                }
            } catch (Exception e) {
                e.printStackTrace();
            } finally {
                // you can close connection here
            }
        }
    }
}
```

oracle date formate is different so we need to use TO_DATE() to convert in oracle style

You should close connection or other resources in finally or use try with resources concept

->you can use finally block to close the connection.

PreparedStatement:

-> PreparedStatement is an interface from java.sql package and which is also used to execute normal queries with IN Parameters.(which means parameterized queries).

->it is the sub interface of statement interface used to execute parameterized query.

->it is faster and performance wise improved then statement.

-> we use preparedStatement() method from 'Connection' interface to create the implementation object of 'PreparedStatement' interface.

Method Signature of preparedStatement():

```
public abstract PreparedStatement prepareStatement (String) throws java.sql.SQLException;
```

ex:- PreparedStatement ps = con.prepareStatement("query-Structure");

->in PreparedStatement only '?' symbol allow , no other symbol allowed for parameter.

->'?' is used to replacing value.(it not effect table name or column name)

->'?' is not allowed in DDL operation.

->'?' is also called parameter or replacement operator or place-resolution operator.

->we need to use setters method to set value.

The following are two important methods of PreparedStatement interface:

(i)executeQuery()

(ii)executeUpdate()

executeQuery(): as we already discuss **executeQuery()** method is used to execute select Queries.

Method Signature of executeQuery():

public abstract ResultSet **executeQuery()** throws java.sql.SQLException;

syntax:

ResultSet rs = ps.executeQuery();

executeUpdate():

=>executeUpdate() method is used to execute Non-Select queries.

Method Signature of executeUpdate():

public abstract int **executeUpdate()** throws java.sql.SQLException;

syntax:

int k = ps.executeUpdate();

```
try {
    Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");

    // Parse the user input string into a Date object
    SimpleDateFormat inputDateFormat = new SimpleDateFormat("dd-MM-yy");
    Date userdate = inputDateFormat.parse(date);

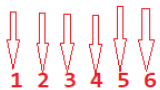
    // Format the date in the way Oracle expects
    SimpleDateFormat oracleDateFormat = new SimpleDateFormat("dd-MMM-yy");
    String oracleFormattedDate = oracleDateFormat.format(userdate);

    PreparedStatement ps = con.prepareStatement("insert into employee values (?, ?, ?, ?, ?, ?)");

    ps.setInt(1, empid);
    ps.setString(2, ename);
    ps.setString(3, jobName);
    ps.setInt(4, sal);
    ps.setString(5, oracleFormattedDate);
    ps.setInt(6, age);

    int inserted = ps.executeUpdate();

    if (inserted > 0) {
        System.out.println("employee data inserted succesfully...");
    } else {
        System.out.println("something went wrong!!");
    }
}
```



```

PreparedStatement ps = con.prepareStatement("insert into employee values (?, ?, ?, ?, ?, ?)");
ps.setInt(1, empid);
ps.setString(2, ename);
ps.setString(3, jobName);
ps.setInt(4, sal);
ps.setString(5, oracleFormattedDate);
ps.setInt(6, age);

```

Task:-fetch employee details based on id

```

public class DatabaseTest3 {

    public static void main(String[] args) throws SQLException {

        Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");

        PreparedStatement ps = con.prepareStatement("select * from employee where empid=?");
        ps.setInt(1, 1112);
        ResultSet rs = ps.executeQuery();

        while(rs.next())
        {
            System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3)+" "+rs.getInt(4)+" "+rs.getDate(5));
        }

    }

}

```

CallableStatement:

->'CallableStatement' is an interface from java.sql package and which is used to execute '**Procedures** and **Functions**' on DataBase product.

define Procedure?

=>procedure is a set-of-queries executed on DataBase product and after execution it will not return any value.

->to execute procedure and function we will use **execute()** method. This method is available in PreparedStatement only. internally CallableStatement inherits PreparedStatement so it is available in it also.

create or replace procedure Proc_name

(para_list) is

begin

query1;

query2;

...

end;

/

define Function?

Function is a set-of-queries executed on DataBase and after execution it will return the value.

Note: Functions in SQL will use 'return' statement to return the value after execution.

structure of Function:

create or replace function Func_name

(para_list) return data_type as var data_type;

begin

query1;

query2;

....

return var;

end;

/

Types of Procedures:

=>Procedures are categorized into two types:

(a)IN Parameter Procedures

(b)OUT Parameter Procedures

(a)IN Parameter Procedures:

=>The Procedures which take the data from JavaPrograms and update DataBase tables are known as 'IN Parameter Procedures'.

->Note:- by default procedure and function parameter are 'IN'.

(b)OUT Parameter Procedures:

=>The Procedures which take the data from DB Tables and send to the Java Programs are known as 'OUT Parameter Procedures'.

```
try {
    Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");

    // Parse the user input string into a Date object
    SimpleDateFormat inputDateFormat = new SimpleDateFormat("dd-MM-yy");
    Date userdate = inputDateFormat.parse(date);

    // Format the date in the way Oracle expects
    SimpleDateFormat oracleDateFormat = new SimpleDateFormat("dd-MM-yy");
    String oracleFormattedDate = oracleDateFormat.format(userdate);

    CallableStatement cs = con.prepareCall("{call myemp(?,?,?,?,?)}");
    cs.setInt(1, empid);
    cs.setString(2, ename);
    cs.setString(3, jobName);
    cs.setInt(4, sal);
    cs.setString(5, oracleFormattedDate);
    cs.setInt(6, age);

    boolean done = cs.execute();
}
```

we are calling procedure which we have create in database we need to write same as above.

ex

```
SQL> create or replace procedure myemp
2  (a number,b varchar2,c varchar2,d number,e date, f number)is
3  begin
4  insert into employee values(a,b,c,d,e,f);
5  end;
6* /
```

by default
parameters are 'in'

```
SQL> create or replace procedure myemp2
2  (a number,b out varchar2,c out varchar2,d out number,e out date,f out number)is
3  begin
4  select ename,job,sal,hiredate,age into b,c,d,e,f from employee where empid=a;
5  end;
6* /
```

here a is 'in' because we are accepting id and rest are 'out' because we want this in our java code.

```
public class Databasetest2 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // EMPID ENAME JOB SAL HIREDATE AGE
        System.out.println("enter employee id");
        int empid = sc.nextInt();
        System.out.println(empid);

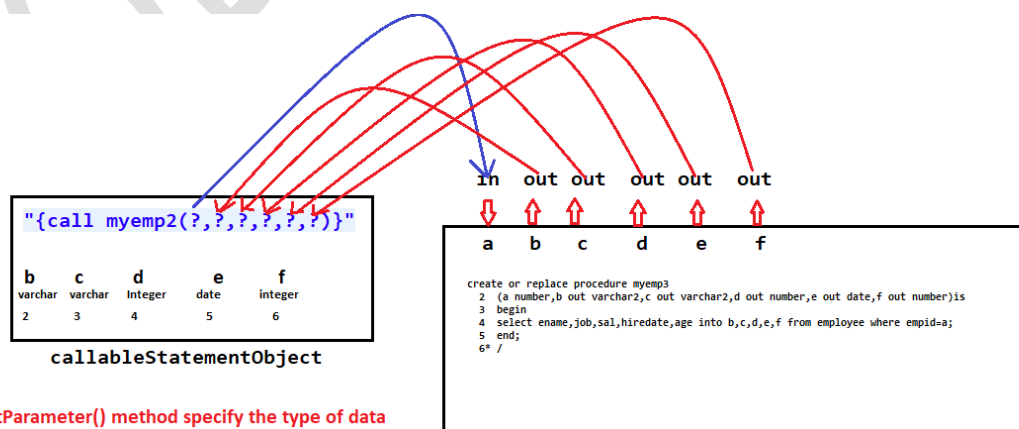
        try {
            Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");

            CallableStatement cs = con.prepareCall("{call myemp2(?,?,?,?)}");
            cs.setInt(1, empid);
            cs.registerOutParameter(2, Types.VARCHAR);
            cs.registerOutParameter(3, Types.VARCHAR);
            cs.registerOutParameter(4, Types.INTEGER);
            cs.registerOutParameter(5, Types.DATE);
            cs.registerOutParameter(6, Types.INTEGER);

            cs.execute();
            System.out.println(cs.getString(2) + " " + cs.getString(3) + " " + cs.getFloat(4) + " " + cs.getDate(5)
                + " " + cs.getInt(6));

        } catch (Exception e) {
```

Note:- you should close resources like connection and all in finally block or use try with resources block....



registerOutParameter() method specify the type of data recorded in Parameter-Index-fields

define registerOutParameter() method?

->registerOutParameter() method specify the type of data recorded in Parameter-Index-fields and registerOutParameter() method is available from 'CallableStatement'.

Task: *Construct and Execute function to retrieve*

Function

```
SQL> create or replace function myempdata
2 (eid number) return number as esal number;
3 begin
4 select sal into esal from employee where empid=eid;
5 return esal;
6 end;
7* /
```

```
public class Databasetest2 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // EMPID ENAME JOB SAL HIREDATE AGE

        System.out.println("enter employee id");
        int empid = sc.nextInt();
        System.out.println(empid);

        try {
            Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");

            CallableStatement cs = con.prepareCall("{call ?:= myempdata(?)}");
            cs.registerOutParameter(1, Types.INTEGER);
            cs.setInt(2, empid);

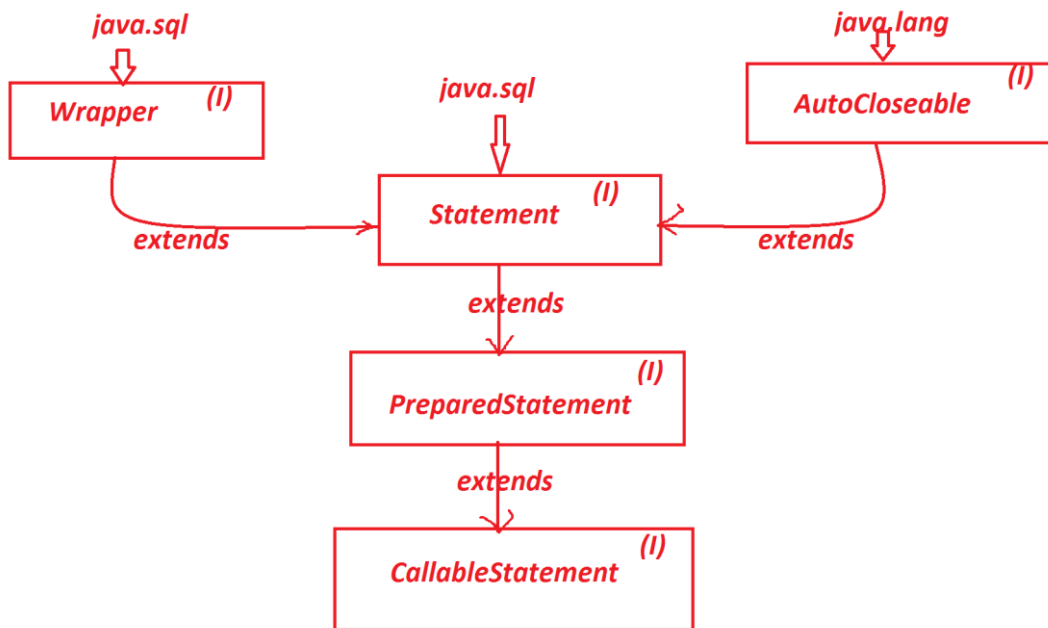
            cs.execute();
            System.out.println(cs.getFloat(1));

        } catch (Exception e) {

            e.printStackTrace();
        } finally {

        }
    }
}
```

Hierarchy of JDBC Statements:



Transaction Management in JDBC:

Define Transaction?

Transaction is a 'set of statements' executed on a resource or resources, by applying ACID properties.

A-Atomicity

C-Consistency

I-Isolation

D-Durability:

A-Atomicity

=>The process of Committing the Transaction after completing all the "SubStatements execution", is known as Atomicity.

Note:

=>Any operation on database must be atomic, which means it must happen

entirely or not at all.

C-Consistency

=>The resources which are selected for Transaction will be same until the transaction is completed is known as Consistency.

I-Isolation

=>The process of running the Transaction independently is known as Isolation.

D-Durability:

=>Once the transaction is committed, the details of Transaction is stored and available for the user, is known as Durability.

define Transaction Management?

->In simple and short words The process of controlling transaction from starting to ending of transaction is known as Transaction Management.

->To deal with the transaction management we use the following methods from 'java.sql.Connection' interface .

- `setAutoCommit()`
- `getAutoCommit()`
- `commit()`
- `rollback()`
- `setSavepoint()`
- `releaseSavepoint()`

setAutoCommit():- actually java application will perform commit operation automatically . so by using this method we can set it to false.

getAutoCommit():-This method is used to know the status of commit operation.

Commit():-we use commit method to update data from instance buffer to database to store permanently.

Rollback():-we use rollback() method to get original state of buffer when the transaction failed. We specify the rollback operation using savepoint.

setSavepoint():-This method is used to set savepoint for rollback operation.

releaseSavepoint():This method is used to delete save point from the buffer.

Task:-transfer amount from one customer account to another customer account.

ACCNO	UNAME	ADHARNO	AMOUNT	TRANSHISTORY
1111	himanshu	123456	4750.23	
2222	rohan	123456	5650	

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    try {
        Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");
        PreparedStatement ps1 = con.prepareStatement("select * from sbi where accno=?");

        PreparedStatement ps2 = con.prepareStatement("update sbi set amount=amount+? where accno=?");
        System.out.println("stop auto committing operation");
        con.setAutoCommit(false);
        Savepoint sp = con.setSavepoint();

        System.out.println("enter account no");
        int haccno = sc.nextInt();
        ps1.setInt(1, haccno);

        ResultSet rs1 = ps1.executeQuery();
        if (rs1.next()) {
            float amount = rs1.getFloat(4);
            System.out.println("enter benificery account no");
            int baccno = sc.nextInt();
            ps1.setInt(1, baccno);
```

```

ResultSet rs2 = ps1.executeQuery();
if (rs2.next()) {
    System.out.println("enter amount to transfer...");
    int transferAmount = sc.nextInt();

    if (transferAmount < amount) {
        System.out.println("updating sender account info");
        ps2.setFloat(1, -transferAmount);
        ps2.setInt(2, haccno);
        int i = ps2.executeUpdate();

        System.out.println("updating receiver account info");
        ps2.setFloat(1, transferAmount);
        ps2.setInt(2, baccno);

        int j = ps2.executeUpdate();

        if (i == 1 && j == 1) {
            System.out.println("transcation succesfull!!");
            con.commit();
        } else {
            System.out.println("Transcation failed");
            con.rollback(sp);
        }
    } else {
        System.out.println("Insufficient fund..");
    }
}

} else {
    System.out.println("No customer with this account no");
}

} else {
    System.out.println("please check your account no");
}

} catch (Exception e) {
    e.printStackTrace();
} finally {
}
}
}

```

Connection Pooling in JDBC:

-> The process of organizing pre-initialized DataBase connections among multiple users is known as Connection Pooling.

-> In connection pooling the users take connection from the pool and uses that connection to perform some operations and return connection to the pool.

Batch Processing in JDBC:

-> The process of collecting multiple queries as a batch and executing on DataBase product is known as '**Batch Processing in JDBC**'.

Batch Processing using 'statement'

=> we use the following methods from 'java.sql.Statement' to perform 'Batch Processing':

- (a) addBatch()
- (b) executeBatch()
- (c) clearBatch()

1)addBatch():

=>This method is used to add the query to the batch.

Method Signature:

public abstract void **addBatch(java.lang.String)**throws java.sql.SQLException;

syntax:

```
stm.addBatch("query");
```

2)executeBatch():

-> This method is used to execute batch on DataBase Product.

Method Signature:

public abstract int[] executeBatch() throws java.sql.SQLException;

syntax:

```
int k[] = stm.executeBatch();
```

3)clearBatch():

-> This method is used to delete(clear) the batch.

Method Signature:

public abstract void **clearBatch()** throws java.sql.SQLException;

syntax:

```
stm.clearBatch();
```

```

public class Databasetest2 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");

            Statement stm = con.createStatement();
            stm.addBatch("insert into sbi values(6666,'nikhil',123456,650,null)"); ➡ 1
            stm.addBatch("insert into boi values(1111,'maxx','35628',700)"); ➡ 2
            int[] batches = stm.executeBatch();
            for (int i = 0; i < batches.length; i++) {
                System.out.println("Data inserted Successfully..!");
            }
            stm.clearBatch();

        } catch (Exception e) {

            e.printStackTrace();
        } finally {

        }

    }
}

```

Batch Processing using 'PreparedStatement'

->we use the following methods to perform Batch processing using PreparedStatement.

(a)addBatch() - PreparedStatement

(b)executeBatch() - Statement

(c)clearBatch() - Statement

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    try {
        Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");

        PreparedStatement ps = con.prepareStatement("insert into sbi (accno,uname,adharno,amount) values(?,?,?,?)");
        ps.setInt(1, 7777);
        ps.setString(2, "rohan");
        ps.setString(3, "1234");
        ps.setFloat(4, 500.30f);

        ps.addBatch();

        ps.setInt(1, 8888);
        ps.setString(2, "mohandas");
        ps.setString(3, "56253");
        ps.setFloat(4, 100.10f);
        ps.addBatch();

        int[] batches = ps.executeBatch();

        for (int i = 0; i < batches.length; i++) {
            System.out.println("Data inserted Successfully..!");
        }
        ps.clearBatch();
    }
}

```

Note:

->'Batch Processing using Statement' we can update multiple DB tables.

->'Batch Processing using PreparedStatement' we can update same DB Table.

Batch processing executes all the queries from batch, in this process execution time is saved and generate high-performance of an application.

->batch processing is also know as **batch update processing**.

Note:-using batch processing we cannot perform '**select**' operation, but we can perform create,insert,update and delete.

Note: using procedure we can perform all DB operation like select,insert,update and delete.

ResultSet

->based on Control over the cursor, the ResultSet Object are Categorized into two types.

- Non-Scrollable ResultSet Object.
- Scrollable ResultSet Object.

Non-Scrolable ResultSet Object:-

->In this non-scrollable resultset Object the cursor is moved only in One direction, which means the cursor moves from top-of the table data to bottom-of the table data.(Top-Bottom)

Scrollable ResultSet Object:-

->In Scrollable ResultSet Object the cursor can be moved in two directions , which means top to bottom of the table data and bottom to top table data.

Note:-we use scrollable resultSet with statement & prepared statement.

Ex:-

Ex:-Statement `stm = con.createStatement(type,mode);`

Ex:-PreparedStatement `ps = con.prepareStatement("query",type,mode);`

Ex:-

Type:

`public static final int TYPE_FORWARD_ONLY=1003`

`public static final int TYPE_SCROLL_INSENSITIVE=1004`

`public static final int TYPE_SCROLL_SENSITIVE=1005`

Mode:

`public static final int CONCUR_READ_ONLY=1007`

`public static final int CONCUR_UPDATABLE=1008`

Note:

'**type**' specifies the direction of the cursor and '**mode**' specifies the action to be performed(read or update).

The following are some Methods used to control cursor on Scrollable

ResultSet object:

afterLast() => Moves the cursor after the Last row

beforeFirst() => Moves the cursor before the First row

previous() => Moves the Cursor in the BackWard Direction

next() => Moves the cursor in the Forward Direction

first() => Moves the cursor to the First row

last() => Moves the cursor to the Last row

absolute(int)=>Moves the cursor to the specified row number

relative(int)=>Moves the cursor from the current position in forward or backward direction by increment or decrement.

```
public class DatabaseTest3 {  
    public static void main(String[] args) throws SQLException {  
        Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");  
        Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,ResultSet.CONCUR_READ_ONLY);  
        ResultSet rs1 = stmt.executeQuery("select * from sbi");  
        System.out.println(".....display data.....");  
        while(rs1.next())  
        {  
            System.out.println(rs1.getInt(1)+" "+rs1.getString(2)+" "+rs1.getString(3)+" "+rs1.getFloat(4));  
        }  
        con.close();  
        stmt.close();  
    }  
}
```

by default result cursor is set in top ...

```
public static void main(String[] args) throws SQLException {  
    Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "c##codemines", "root");  
    Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,ResultSet.CONCUR_READ_ONLY);  
    ResultSet rs1 = stmt.executeQuery("select * from sbi");  
    System.out.println(".....display data in reverse .....");  
    rs1.afterLast();  
    while(rs1.previous())  
    {  
        System.out.println(rs1.getInt(1)+" "+rs1.getString(2)+" "+rs1.getString(3)+" "+rs1.getFloat(4));  
    }  
    con.close();  
    stmt.close();  
}
```

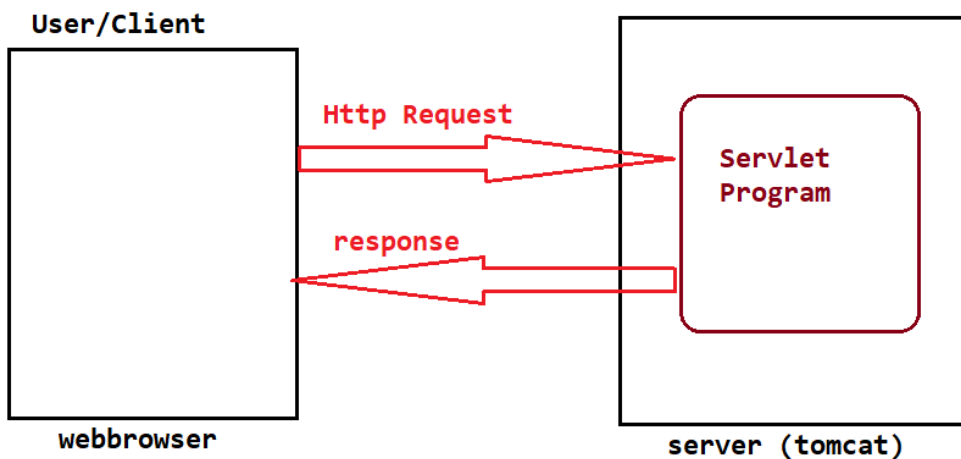
this method set cursor at last

from last we move cursor previous means bottom to top to display records in reverse

SERVLET PROGRAMMING

->In simple terminology we can say a independent java program running in server environment and accepting the request from the client through webbrowser is know as “**servlet programming**”

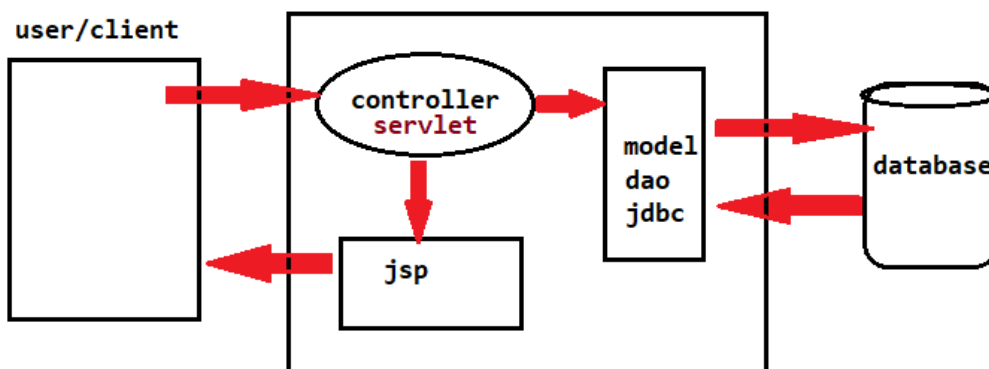
Server:- server is also a code running on different machine or in same machine (localhost) which accepts the request from the client and generating response.



Web Applications:

->The applications which are executing in Web Environment or Internet Environment are known as Web Applications.

Web application



->we use servlet to create controller of the web-application.

->These Web Application are executed in 'Web Container'.

->WebContainer is available from WebServers and Application Servers.

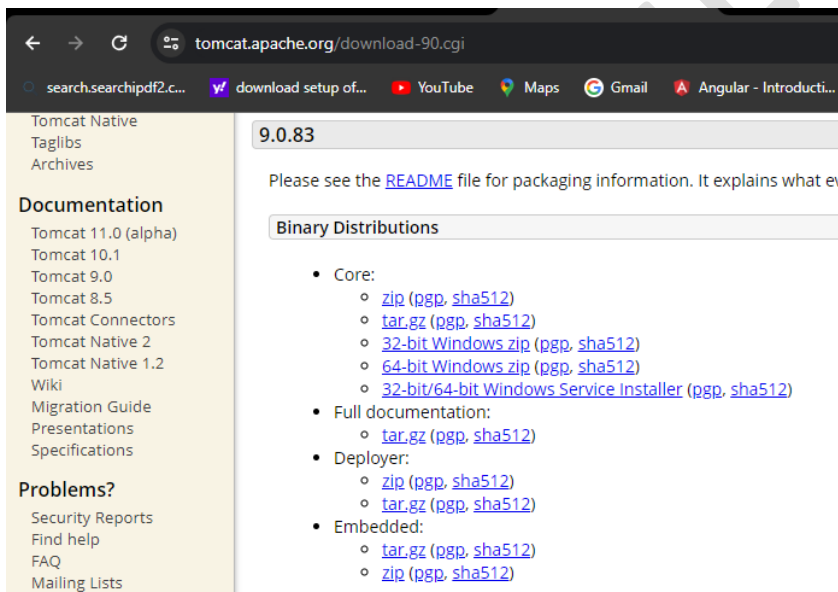
->Web Container internally has two SubContainers.

1.Servlet Container. (Catelina)

2.JSP Container.(Jasper)

Installing Tomcat Server

Step1.Download tomcat9.xx version which is compatable with jdk1.8 and above.



Step2:- install tomcat server.

Step3:-start the tomcat server.

Goto 'bin' and Click on 'startup' or 'Tomcat9w' to start the Tomcat Server

step-4 : Check the Tomcat Server is responding to WebBrowser or not,using the following address: **http://localhost:8081**

File Name	Date/Time	Type	Size
catalina.bat	28/06/2021 10:46 AM	Windows Batch File	17 KB
catalina.sh	28/06/2021 10:46 AM	Shell Script	25 KB
catalina-tasks.xml			
ciphers.bat			
ciphers.sh			
commons-daemon.jar			
configtest.bat			
configtest.sh			
daemon.sh			
digest.bat			
digest.sh			
makebase.bat			
makebase.sh			
service.bat			
setclasspath.bat			
setclasspath.sh			
shutdown.bat			
shutdown.sh			
startup.bat			
startup.sh			
tcnative-1.dll			
tomcat9.exe			
tomcat9w.exe			
tomcat-juli.jar			
tool-wrapper.bat			
tool-wrapper.sh			


Tomcat

```

28-Nov-2023 22:40:07.126 INFO [main] org.apache.catalina.startup.
ctory [C:\apache-tomcat-9.0.50\webapps\docs]
28-Nov-2023 22:40:07.267 INFO [main] org.apache.catalina.startup.
directory [C:\apache-tomcat-9.0.50\webapps\docs] has finished in
28-Nov-2023 22:40:07.267 INFO [main] org.apache.catalina.startup.
ctory [C:\apache-tomcat-9.0.50\webapps\examples]
28-Nov-2023 22:40:11.533 INFO [main] org.apache.catalina.startup.
directory [C:\apache-tomcat-9.0.50\webapps\examples] has finished
28-Nov-2023 22:40:11.533 INFO [main] org.apache.catalina.startup.
ctory [C:\apache-tomcat-9.0.50\webapps\host-manager]
28-Nov-2023 22:40:11.736 INFO [main] org.apache.catalina.startup.
directory [C:\apache-tomcat-9.0.50\webapps\host-manager] has fini
28-Nov-2023 22:40:11.736 INFO [main] org.apache.catalina.startup.
ctory [C:\apache-tomcat-9.0.50\webapps\manager]
28-Nov-2023 22:40:11.923 INFO [main] org.apache.catalina.startup.
directory [C:\apache-tomcat-9.0.50\webapps\manager] has finished
28-Nov-2023 22:40:11.923 INFO [main] org.apache.catalina.startup.
ctory [C:\apache-tomcat-9.0.50\webapps\myservproject]
28-Nov-2023 22:40:12.033 INFO [main] org.apache.catalina.startup.
directory [C:\apache-tomcat-9.0.50\webapps\myservproject] has fir
28-Nov-2023 22:40:12.033 INFO [main] org.apache.catalina.startup.
ctory [C:\apache-tomcat-9.0.50\webapps\ROOT]
28-Nov-2023 22:40:12.173 INFO [main] org.apache.catalina.startup.
directory [C:\apache-tomcat-9.0.50\webapps\ROOT] has finished in
28-Nov-2023 22:40:12.189 INFO [main] org.apache.coyote.AbstractPr
28-Nov-2023 22:40:13.205 INFO [main] org.apache.catalina.startup.

```

localhost:8081

search.searchipdf2.c... download setup of... YouTube Maps Gmail Angular - Introdu... Firebase CRUD Web... Oracle Tablespace @FreshersJobsUpda...

Home Documentation Configuration Examples Wiki Mailing Lists Find Help

Apache Tomcat/9.0.50



If you're seeing this, you've successfully installed Tomcat. Congratulations!

Recommended Reading:

- [Security Considerations How-To](#)
- [Manager Application How-To](#)
- [Clustering/Session Replication How-To](#)

Developer Quick Start

- [Tomcat Setup](#)
- [First Web Application](#)
- [Realms & AAA](#)
- [JDBC DataSources](#)
- [Examples](#)
- [Servlet Specifications](#)
- [Tomcat Versions](#)

Managing Tomcat

For security, access to the [manager webapp](#) is restricted. Users are defined in:

\$CATALINA_HOME/conf/tomcat-users.xml

In Tomcat 9.0 access to the manager application is split between different users.

[Read more...](#)

Documentation

[Tomcat 9.0 Documentation](#)

[Tomcat 9.0 Configuration](#)

[Tomcat Wiki](#)

Find additional important configuration information in:

Getting Help

[FAQ and Mailing Lists](#)

The following mailing lists are available:

- [tomcat-announce](#)
Important announcements, releases, security vulnerability notifications. (Low volume).
- [tomcat-users](#)

[Server Status](#)
[Manager App](#)
[Host Manager](#)

Step5:-to stop it goto 'bin' and click on 'shutdown'.

 digest.bat	28/06/2021 10:46 AM	Windows Batch File	3 KB
 digest.sh	28/06/2021 10:46 AM	Shell Script	2 KB
 makebase.bat	28/06/2021 10:46 AM	Windows Batch File	4 KB
 makebase.sh	28/06/2021 10:46 AM	Shell Script	4 KB
 service.bat	28/06/2021 10:46 AM	Windows Batch File	9 KB
 setclasspath.bat	28/06/2021 10:46 AM	Windows Batch File	4 KB
 setclasspath.sh	28/06/2021 10:46 AM	Shell Script	4 KB
 shutdown.bat	28/06/2021 10:46 AM	Windows Batch File	2 KB
 shutdown.sh	28/06/2021 10:46 AM	Shell Script	2 KB
 startup.bat	28/06/2021 10:46 AM	Windows Batch File	2 KB
 startup.sh	28/06/2021 10:46 AM	Shell Script	2 KB
 tcnative-1.dll	28/06/2021 10:46 AM	Application exten...	2,546 KB
 tomcat9.exe	28/06/2021 10:46 AM	Application	128 KB
 tomcat9w.exe	28/06/2021 10:46 AM	Application	118 KB

Servlet API

->"**javax.servlet**" package is the Servlet Api.

->"**javax.servlet.Servlet**" interface is the root of Servlet API.

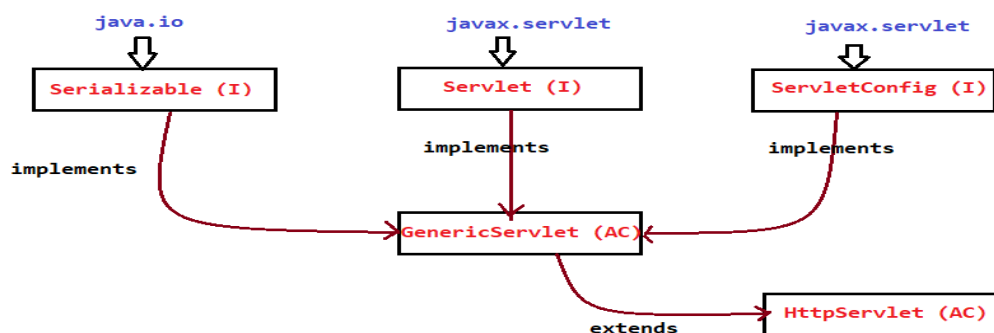
->The Following are the methods from "**Servlet**" interface.

- Init()
- service()
- destroy()
- getServletConfig()
- getServletInfo()

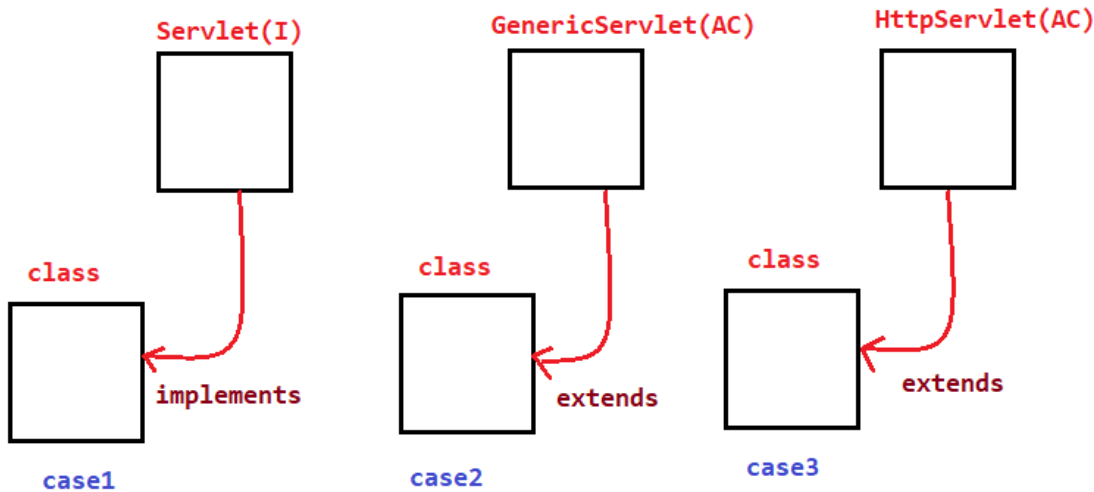
Note:- if you are using recent version then you will get all this inside jakarta package.

Note:-init(),service() and destroy() methods are the Life-cycle methods and executed automatically in the same Order.

Hierarchy of Servlet API:



-> In the process of Constructing ServletProgram the user defined class must be implemented from '**Servlet**' or extended from '**GenericServlet**' or extended from '**HttpServlet**'.



Steps to create First Servlet Program which implements Servlet Api

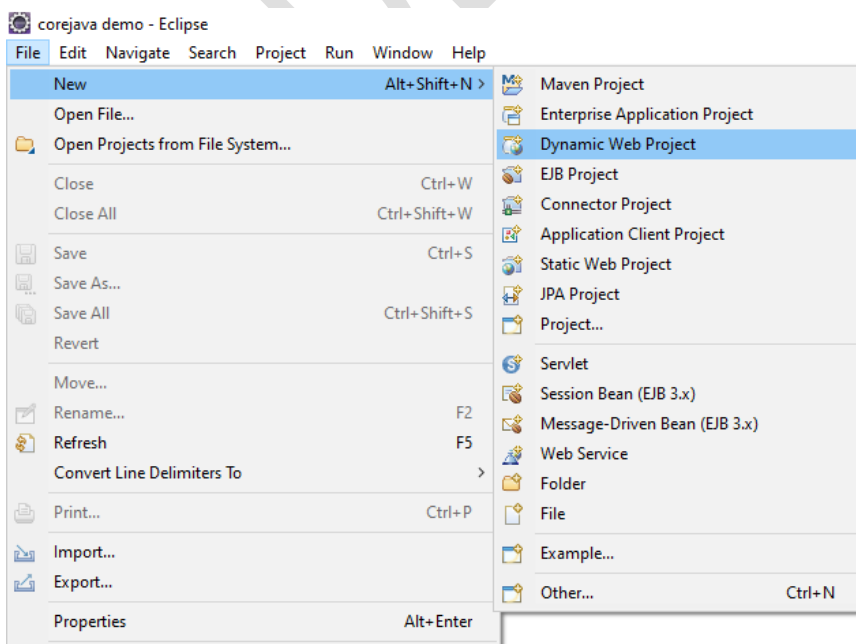
Step1:- Open eclipse IDE select workspace and launch .

Step2:- create dynamic web project.

File->new ->dynamic web project and click on next->name the project and finish.

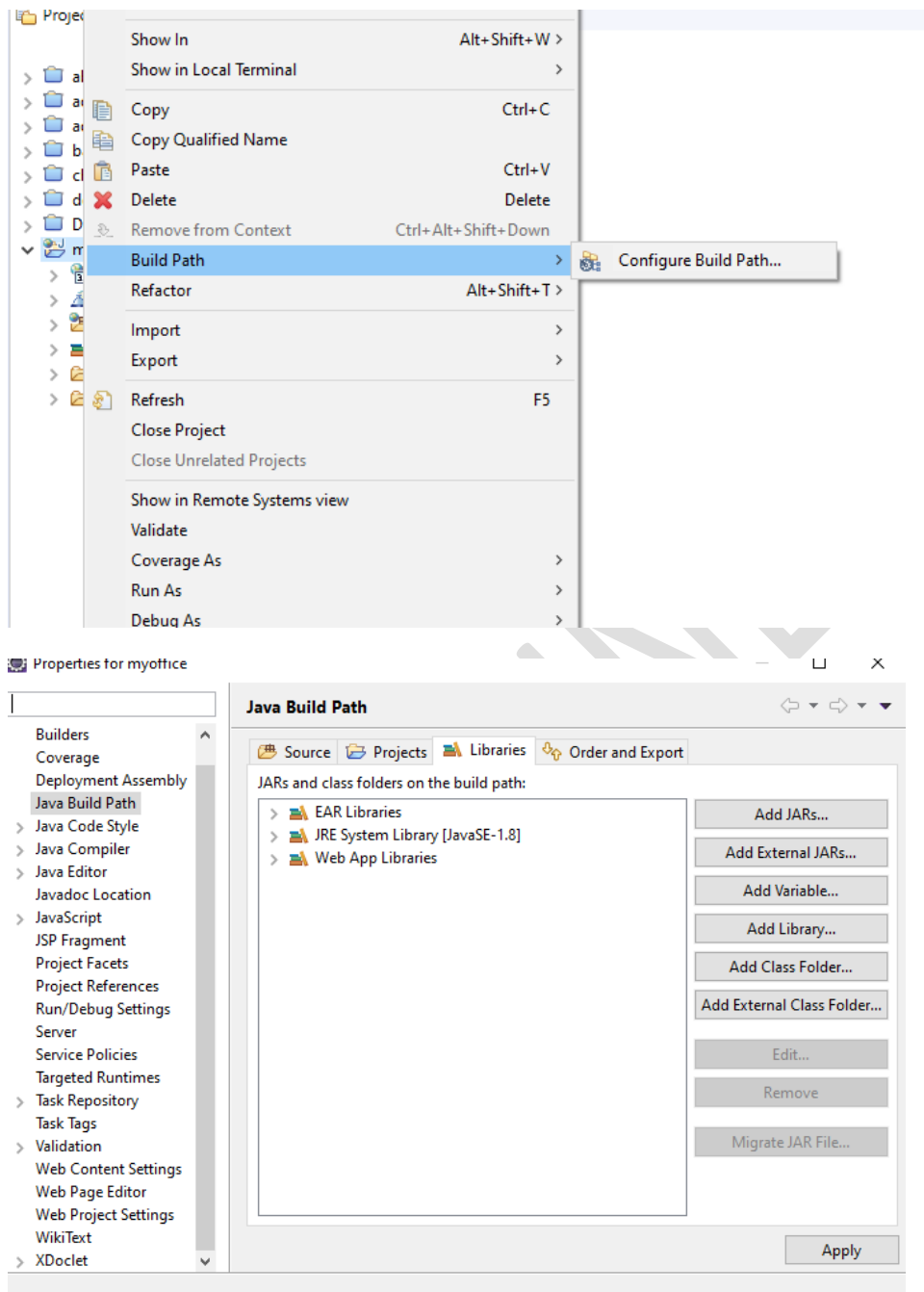
If dynamic web project option is not displaying directly then

File->new->project->web->dynamic web project->name the project and finish.

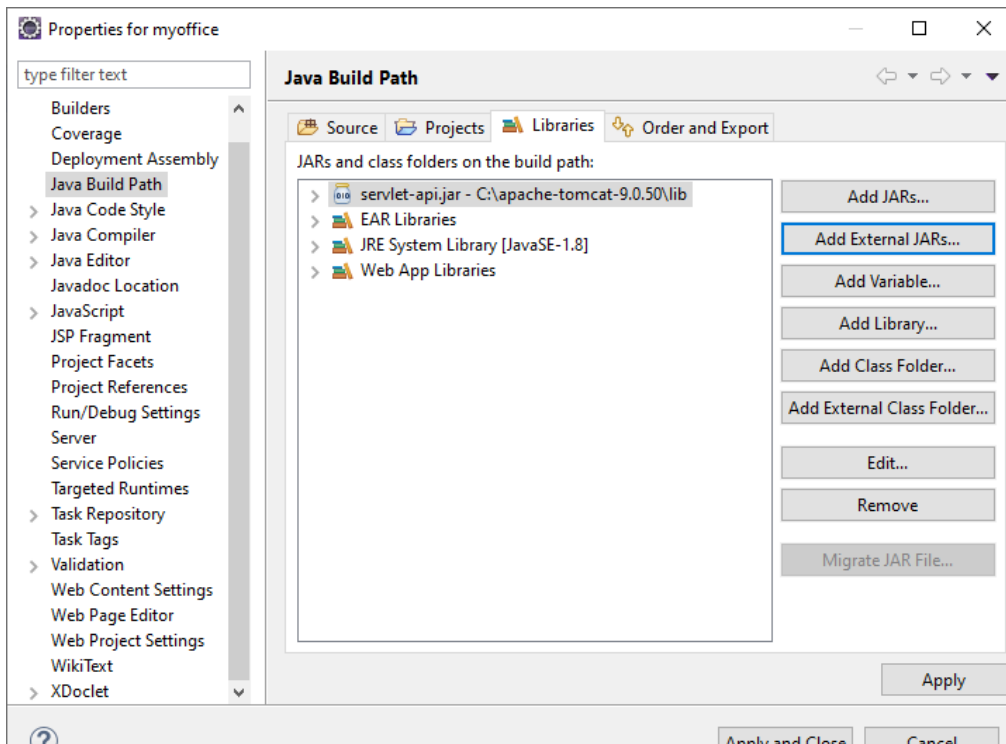


Step3:- add Servlet-api.jar because we are creating servlet program.All Servlet related predefined classes and interface available in that jar.

->select project go to build path ->configure build path->libraries->add external jar.



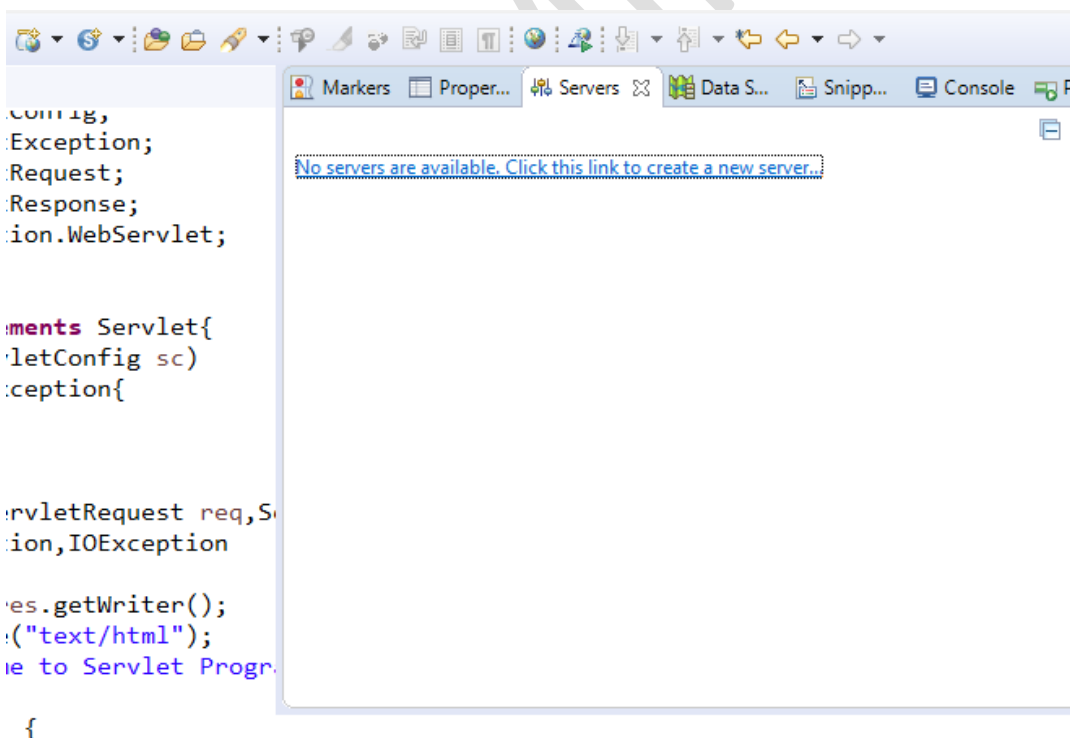
->go to tomcat folder (in my case it is installed in **c drive**) and select **Lib** folder inside that select **servlet-api.jar** file.



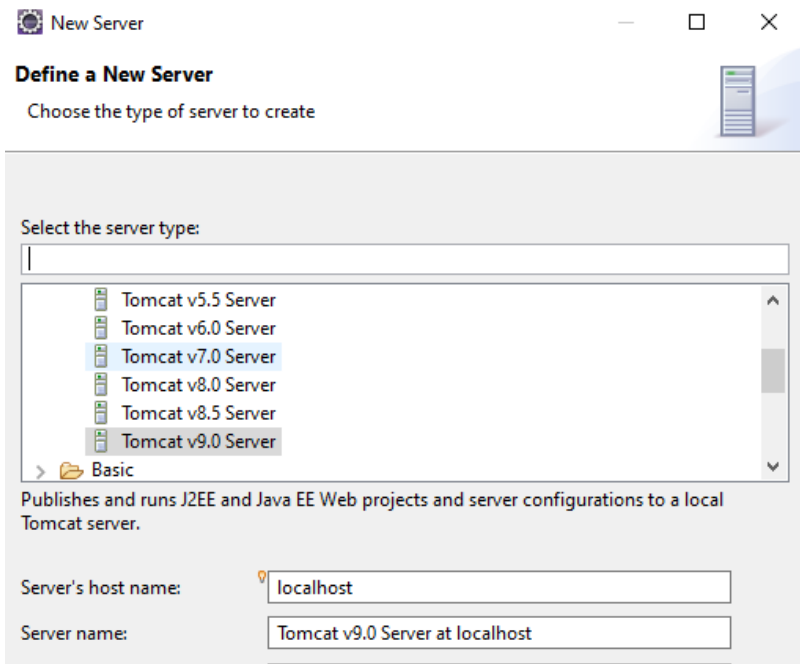
Note:- while adding servlet-api.jar file . please take care that if you are getting option of class path inside libraries then you need to select class path and add there .in my case no classpath inside libraries so using directly

Now we have added servlet api in our project we are ready to create servlet program.

Step4:- Now we need to add Tomcat server to Eclipse IDE because servlet is a server side program to execute it we need server environment .

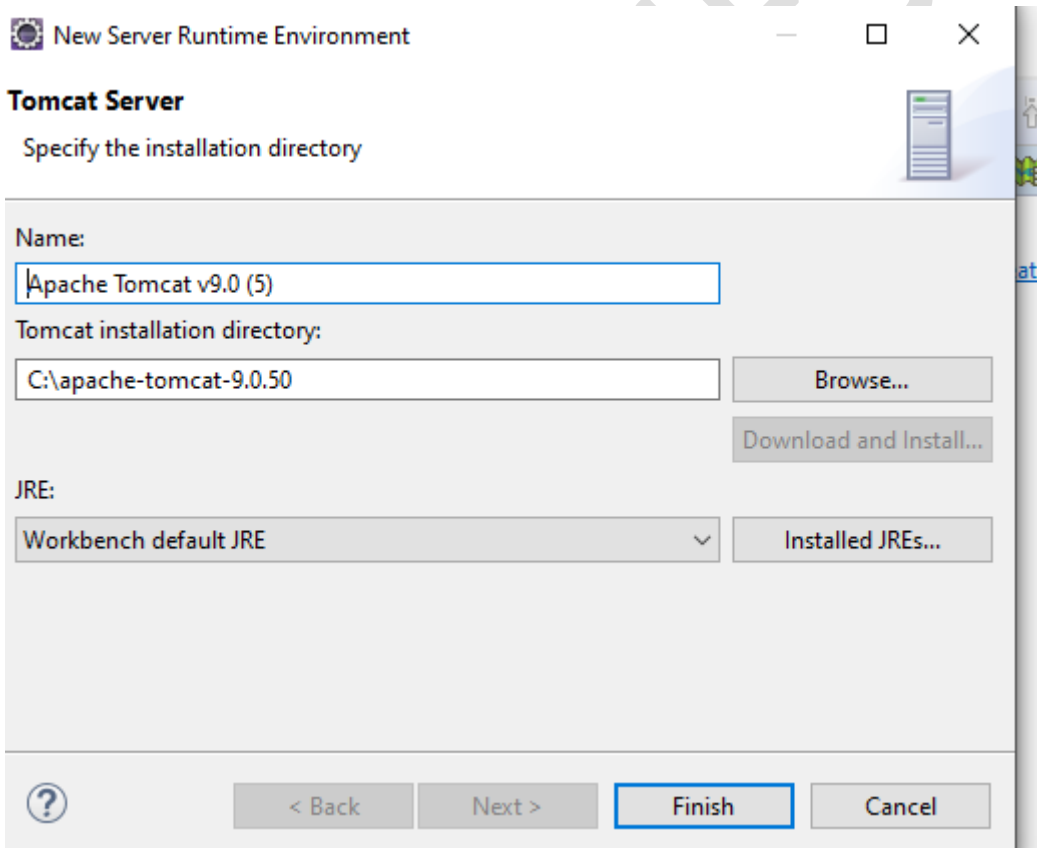


->select console and choose server option and click on given blue link.



Select tomcat9

->now select tomcat folder (in my case it is present in c drive)



Step5:-now to execute program just right click and run as server.

```

@WebServlet("/xyz")
public class MyServlet implements Servlet {

    public void init(ServletConfig sc) throws ServletException {
        // .....
    }

    public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException {
        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");
        pw.println("Welcome to Servlet Programming...");
    }

    public void destroy() {
        // NoCode
    }

    public String getServletInfo() {
        return "FisrtServlet";
    }

    public ServletConfig getServletConfig() {
        return this.getServletConfig();
    }
}

```

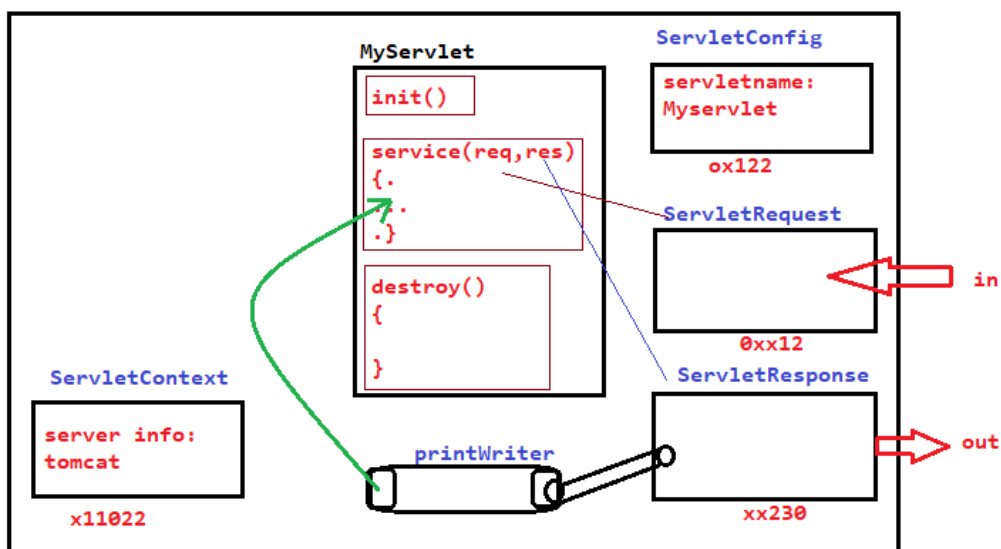
xyz is request point for this servlet

we have overridden all the abstract method present in Servlet Interface.

init,service,destroy are the servlet life cycle method invoke automatically

Run:- http://localhost:8081/Servlet_App1/xyz

Explanation:-



->above diagram is explaining every thing.this big box is web container means as we run our program web container came into picture. Inside webcontainer lots of object are created few are most important to understand.

As webcontainer started ServletContext object is created which holds server information.

->after that our servlet program loads and as our servlet program loads one ServletConfig object created which hold our Servlet information like name and all. And now our servlet execution started . as we know in our servlet init(),service(),destroy() methods are servlet life-cycle method which invokes automatically.and two more new object will be created i.e request, response ...all these work will be done by web container just we need to use it.

ServletContext:

=>ServletContext is an interface from 'javax.servlet' package and which is instantiated automatically when the web App is deployed into Server.

=>This ServletContext object is loaded with Server Information.

ServletConfig:

=>ServletConfig is an interface from 'javax.servlet' package and which is instantiated automatically when the Servlet program is loaded for execution.

=>This ServletConfig object is loaded with Servlet_name

ServletRequest:

=>ServletRequest is an interface from 'javax.servlet' package and which is instantiated automatically while service() method execution.

=>This ServletRequest object will hold the data submitted from HTML forms.

ServletResponse:

=>ServletResponse is an interface from 'javax.servlet' package and which is instantiated automatically while service() method execution.

=>ServletResponse object will hold the data which we sending as output(response).

define getWriter() method?

->getWriter() method is available from 'ServletResponse' and which is used to create object for PrintWriter class.

syntax:

```
PrintWriter pw = res.getWriter();
```

->this PrintWriter object is internally linked to ServletResponse object.

setContentType() method ?

->this method is available in 'ServletResponse' and which specify the type of data sending as response.

syntax:

```
res.setContentType("text/html");
```

program2:-Read data from html form

create html file in webcontent folder.

```
<html>
<head>
<title>home</title>
</head>
<body>
<form action="user" method="POST">
<input type="text" name="uname" placeholder="enter name"/>
<input type="number" name="uage" placeholder="enter age"/>
<input type="text" name="uadd" placeholder="enter address"/>
<input type="submit" value="register"/>

</form>

</body>
</html>
```

->now create servlet program which accept request came from this form submission.

Note:- it is mandatory to provide "name" attribute in form elements . because based on that only we catch form element in servlet program.

```

@WebServlet("/user")
public class MyServlet implements Servlet {
    public void init(ServletConfig sc) throws ServletException {
        // .....
    }

    public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException {
        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");
        pw.println("Welcome to Servlet Programming...");

        String uname = req.getParameter("uname");
        String age = req.getParameter("uage");
        String add = req.getParameter("uadd");

        PrintWriter out = res.getWriter();
        out.println("<h1>user details</h1>");
        out.println("<h3>name"+uname+"</h3>");
        out.println("<h3>age"+age+"</h3>");
        out.println("<h3>address "+add+"</h3>");

    }

    public void destroy() {
        // NoCode
    }
}

```

http://localhost:8081/servletdemo/input.html

input.html

```

<html>
<head>
<title>home</title>
</head>
<body>
<form action="user" method="POST">
<input type="text" name="uname" placeholder="enter name"/>
<input type="number" name="uage" placeholder="enter age"/>
<input type="text" name="uadd" placeholder="enter address"/>
<input type="submit" value="register"/>
</form>
</body>
</html>

```

```

@WebServlet("/user")
public class MyServlet implements Servlet {
    public void init(ServletConfig sc) throws ServletException {
        // .....
    }

    public void service(ServletRequest req, ServletResponse res)
        throws ServletException, IOException {

        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");
        pw.println("Welcome to Servlet Programming...");
        String uname = req.getParameter("uname");
        String age = req.getParameter("uage");
        String add = req.getParameter("uadd");

        PrintWriter out = res.getWriter();
        out.println("<h1>user details</h1>");
        out.println("<h3>name"+uname+"</h3>");
        out.println("<h3>age"+age+"</h3>");
        out.println("<h3>address "+add+"</h3>");

    }
}

```

->in servlet program we use printwriter object and use it's println() method to send response.

What is getParameter() method?

->getParameter() method is from 'ServletRequest' and which is used to get the data from ServletRequest object into Servlet program.

Syn:- String val = req.getParameter("para_name");

Note:

=>Servlet-url-pattern is holded by

- (i)Annotation ex:- @WebServlet("/url-pattern")
- (ii)xml file(web.xml)

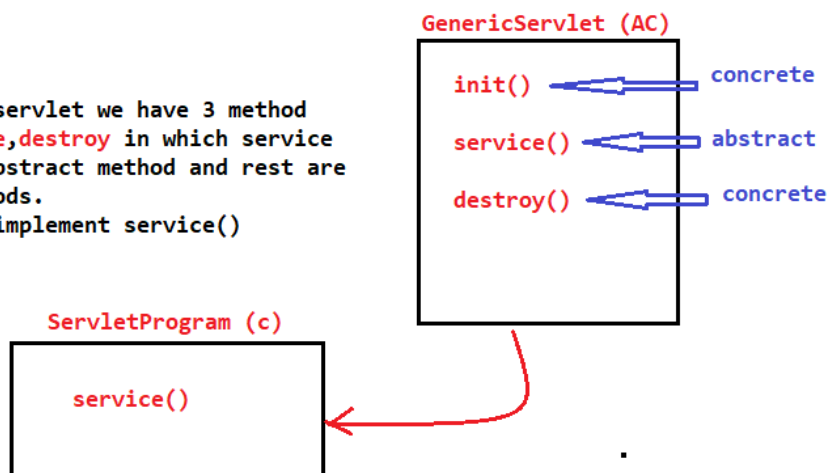
As off now we have created one servlet program by implementing Servlet. Means by using '**javax.servlet.Servlet**' interface. But this interface is not preferable because the objects are Not-Serializable which means Object cannot travel over network.

->The limitation of 'Servlet' interface can be Overcomed using '**GenericServlet**' and '**HttpServlet**'.

->The Object generated through GenericServlet & HttpServlet are serializable.

Constructing Servlet programs using 'GenericServlet'

in generic servlet we have 3 method **init,service,destroy** in which service method is abstract method and rest are normal methods.
we need to implement service()



GenericServlet program..

```

input.html  MyServlet.java
1<html>
2<head>
3  <title>home</title>
4  </head>
5<body>
6<form action="user" method="POST">
7  <input type="text" name="uname" placeholder="enter name"/>
8  <input type="number" name="uage" placeholder="enter age"/>
9  <input type="text" name="uadd" placeholder="enter address"/>
10 <input type="submit" value="register"/>
11
12 </form>
13
14 </body>
15 </html>

@WebServlet("/user")
public class MyServlet extends GenericServlet {
    public void init(ServletConfig sc) throws ServletException {
        // .....
    }

    public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException {
        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");
        pw.println("Welcome to GenericServlet Programming...");
        String uname = req.getParameter("uname");
        String age = req.getParameter("uage");
        String add = req.getParameter("uadd");

        PrintWriter out = res.getWriter();
        out.println("<h1>user details</h1>");
        out.println("<h3>name"+uname+"</h3>");
        out.println("<h3>age"+age+"</h3>");
        out.println("<h3>address "+add+"</h3>");
    }

    public void destroy() {
        // NoCode
    }

    public String getServletInfo() {
        return "FisrtServlet";
    }
}

```

RequestDispatcher

->RequestDispatcher is an interface from 'javax.servlet' package and which provides the following methods to establish communication b/w servlet programs.

Methods of requestDispatcher.

- forward()
- Include()

Note:- RequestDispatcher is an interface which means we cannot create its object .

->We use '**getRequestDispatcher()**' method from '**ServletRequest**' interface to create the implementation object of 'RequestDispatcher' . getRequestDispatcher() method is factory method.

Method Signature of getRequestDispatcher():

Public abstract RequestDispatcher getRequestDispatcher(java.lang.String);

Syn:-

```
RequestDispatcher rd = req.getRequestDispatcher("url-patter/jsp/html")
```

↓
this method take one
parameter ex:-servlet
name,jsp name,html..etc

Forward Communication Process.

->in simple words in forward communication process FirstServlet will take the request and Forward the request to the SecondServlet. In this process the response is generated by second servlet.

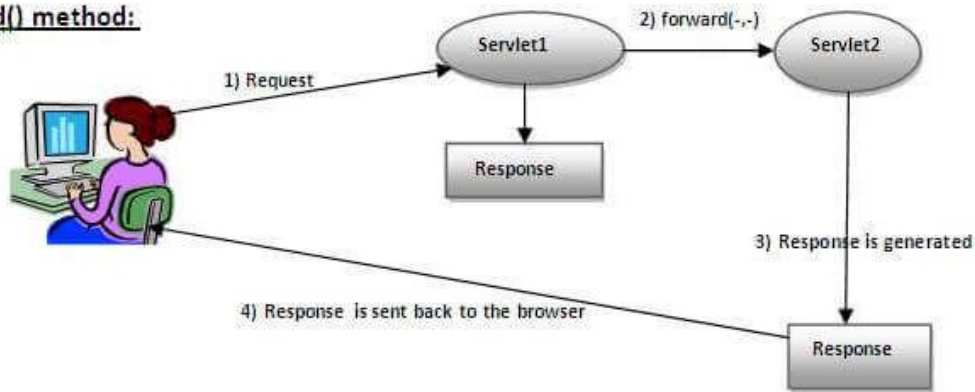
->The second servlet can also be replaced with JSP file or HTML file.

->we use forward() method from RequestDispatcher to establish forward communication.

syn

```
RequestDispatcher rd = req.getRequestDispatcher("html/jsp/servlet");  
rd.forward(req, res);
```

forward() method:



Include Communication process:

->in simple words in include communication process the response is generated from FirstServlet , but the generated response is added with the response of secondServlet .

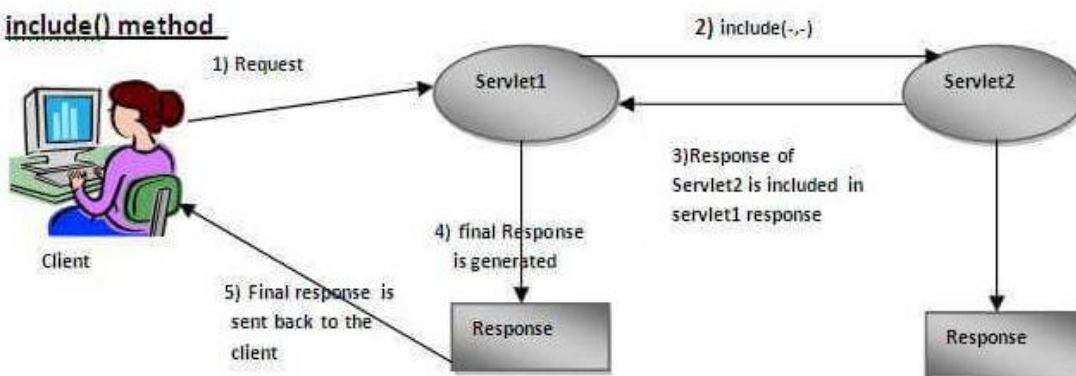
->in place of second servlet jsp/html can be used.

->we use include() method from RequestDispatcher to perform include communication process.

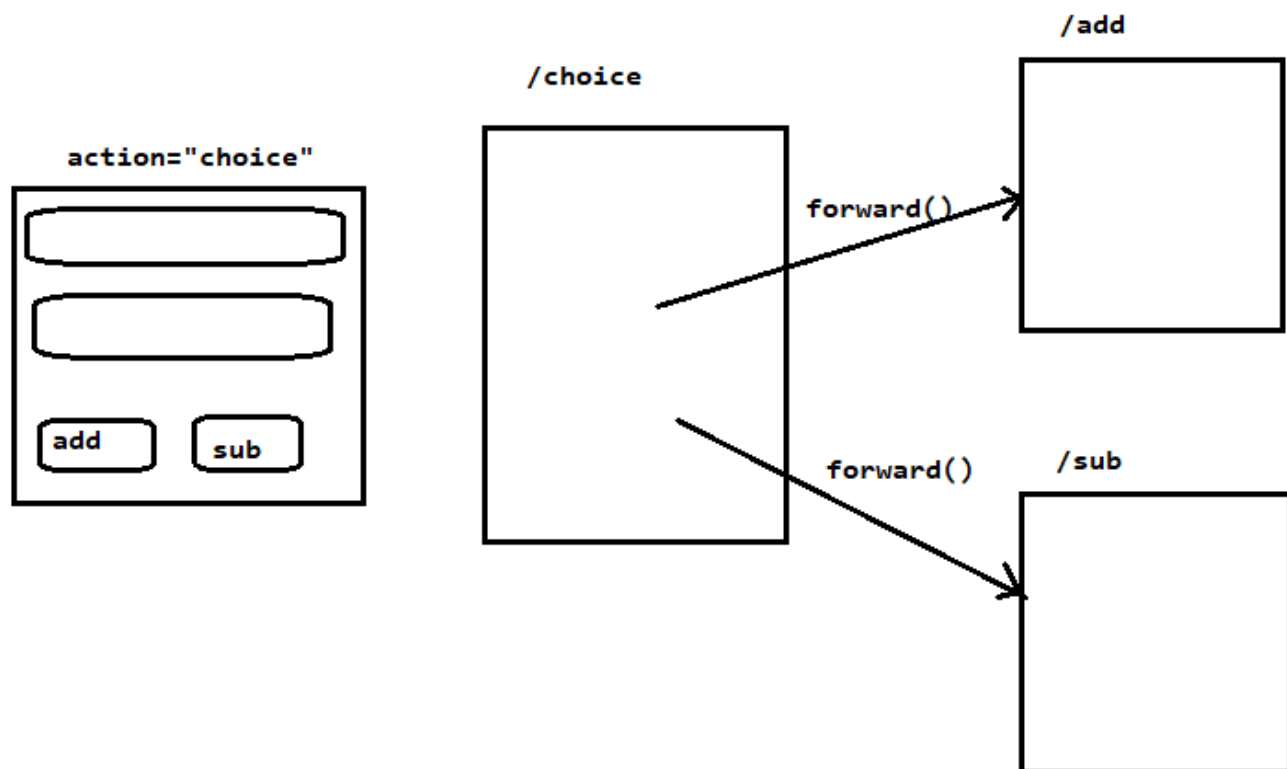
syntax:

rd.include(req,res);

include() method



Ex_Application:(Demonstrating forward() and include() methods)



input.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<form action="choice" method="post">
Enter the Value1:<input type="text" name="v1"><br>
Enter the Value2:<input type="text" name="v2"><br>
<input type="submit" value="Add" name="s1">
<input type="submit" value="Sub" name="s1">
</form>
</body>
</html>
```

->

```

@WebServlet("/choice")
public class ChoiceServlet extends GenericServlet {

    public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException {
        String s1 = req.getParameter("s1");
        if (s1.equals("Add")) {
            RequestDispatcher rd = req.getRequestDispatcher("add");
            rd.forward(req, res);
        } else {
            RequestDispatcher rd = req.getRequestDispatcher("sub");
            rd.forward(req, res);
        }
    }

}

}

```

```

@WebServlet("/add")
public class Add extends GenericServlet {

    public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException {
        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");
        try {
            int v1 = Integer.parseInt(req.getParameter("v1"));
            int v2 = Integer.parseInt(req.getParameter("v2"));
            int v3 = v1 + v2;
            pw.println("Sum: " + v3 + "<br>");
        } catch (Exception e) {
            pw.println("Enter only Integer Values...<br>");
        }

        RequestDispatcher rd = req.getRequestDispatcher("input.html");
        rd.include(req, res);
    }

}

}

```

```

@WebServlet("/sub")
public class Sub extends GenericServlet {

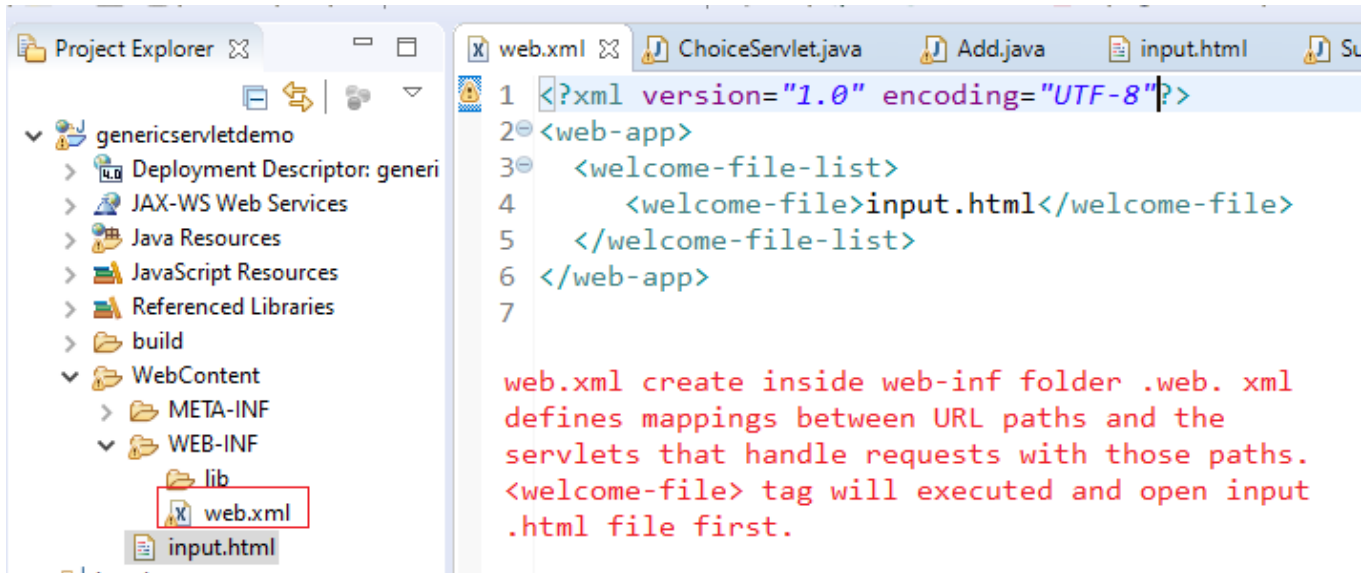
    public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException {
        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");
        try {
            int v1 = Integer.parseInt(req.getParameter("v1"));
            int v2 = Integer.parseInt(req.getParameter("v2"));
            int v3 = v1 - v2;
            pw.println("Sub: " + v3 + "<br>");
        } catch (Exception e) {
            pw.println("Enter only Integer Values...<br>");
        }

        RequestDispatcher rd = req.getRequestDispatcher("input.html");
        rd.include(req, res);
    }

}

}

```



ServletContext:

- >ServletContext is an interface from javax.servlet package and which is instantiated automatically when application is deployed.
- >there is only one ServletContext object per web application.
- >if any information is shared in many servlet then servlet context is better place.
- >The information in ServletContext object can be used by all the servlets in the web application.
- >we use getServletContext() method to access ServletContext Object.
- >this getServletContext() method is available from **GenericServlet** , **ServletRequest** , **ServletConfig** and **HttpSession**.

Method Signature of getServletContext() from ServletRequest:

syntax:

```
ServletContext sct = req.getServletContext();
```

=>we use <context-param> tag in web.xml to initialize parameter-values to ServletContext object.

syntax:

<web-app>

<context-param>

<param-name> name </param-name>

<param-value> value </param-value>

</context-param>



*we can set value in
servlet context using
this tag*

<servlet-mapping>

<servlet-name> name </servlet-name>

<url-pattern> url </url-pattern>

</servlet-mapping>

<servlet>

<servlet-name> name </servlet-name>

<servlet-class> class </servlet-class>

</servlet>

*when we hit url request goes to server and in server
mapping tag will map this URL to particular servlet name
and <servlet> tag contains that class which that servlet
belongs.*

</web-app>

=>we use 'getInitParameter()' method to access initialized parameter from the ServletContext object.

syntax:

String var = sct.getInitParameter("para_name");

ServletConfig:

=>ServletConfig is an interface from 'javax.servlet' package and which is instantiated automatically when the Servlet program is loaded on to Web Container for execution.

=>This ServletConfig object is loaded with Servlet_name.

=>Every Servlet program will have its own ServletConfig object.

=>we use getServletConfig() method to access ServletConfig object into Servlet program.

=>This getServletConfig() method is available from 'GenericServlet'.

syntax:

```
ServletConfig sc = this.getServletConfig();
```

=>we use <init-param> sub-tag part of <servlet> tag to initialize parameter-values into ServletConfig object.

syntax:

<web-app>

<servlet>

<servlet-name> name </servlet-name>

<servlet-class> class </servlet-class>

<init-param>

<param-name> name </param-name>

<param-value> value </param-value>

</init-param>

</servlet>

</web-app>

ServletConfig is separate for every servlet program so in web.xml we use it inside <servlet> tag

wt is the diff b/w

- (i)getParameter()
- (ii)getInitParameter()

=>getParameter() method is used to access the parameters from ServletRequest

object.

=>getInitParameter() method is used to access the parameters from ServletContext and ServletConfig objects.

input.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Input form</title>
</head>
<body>
  <form action="home" method="post">
    UserName:<input type="text" name="uname"><br>
    <input type="submit" value="sub">
  </form>
</body>
</html>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app>
  <servlet>
    <servlet-name>homeserv</servlet-name>
    <servlet-class>test.demoServlet</servlet-class>
    <init-param>
      <param-name>total</param-name>
      <param-value>500</param-value>
    </init-param>
  </servlet>
  <servlet-mapping>
    <servlet-name>homeserv</servlet-name>
    <url-pattern>/home</url-pattern>
  </servlet-mapping>
  <welcome-file-list>
    <welcome-file>input.html</welcome-file>
  </welcome-file-list>
</web-app>
```

```

public class DemoServlet extends GenericServlet{

    public void service(ServletRequest req,ServletResponse res) throws ServletException,IOException
    {

        PrintWriter pw = res.getWriter();

        res.setContentType("text/html");

        String uName = req.getParameter("uname");

        pw.println("Welcome User "+uName);

        pw.println("<br>-----ServletConfig-----");

        ServletConfig sc = this.getServletConfig();

        pw.println("<br>ServletName:"+sc.getServletName());

        pw.println("<br>ConfigValue:"+sc.getInitParameter("total "));

    }

}

```

"attribute" in Servlet programming:

=>"attribute" is a variable in Servlet programming which can be added to **ServletContext object** , **ServletRequest Object** and **HttpSession object**.

->There are some methods related to 'attribute'.

- setAttribute()
- getAttribute()
- removeAttribute()

setAttribute():- this method is used to set data (attribute) to the objects like **HttpSession,ServletContext,ServletRequest** from java code instead of web.xml.

getAttribute():-This method is used to get the attribute from the objects.

removeAttribute():-This method is used to remove attribute from the object.

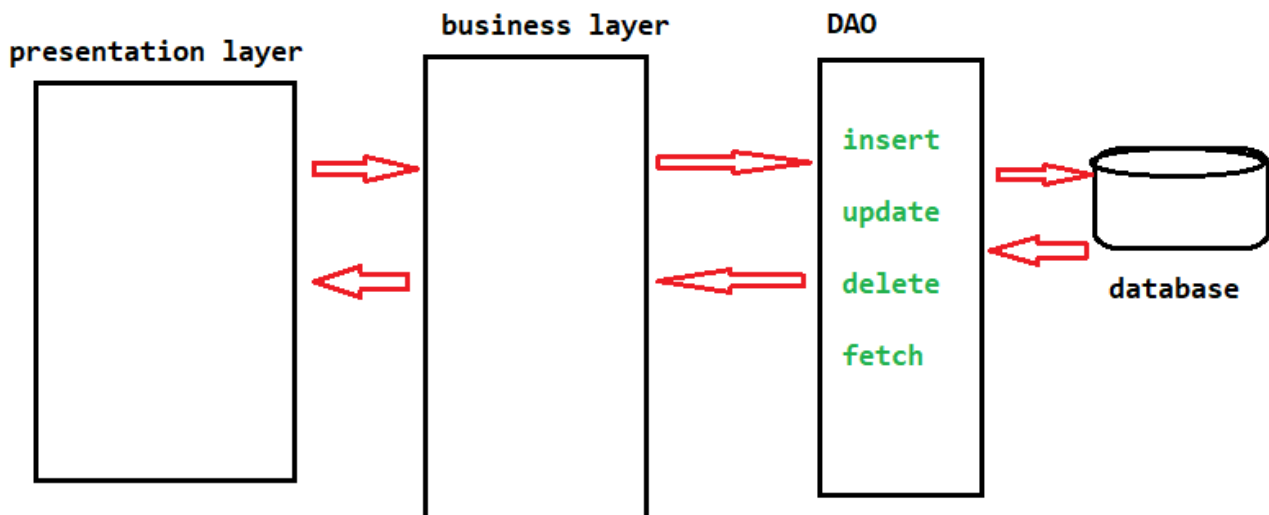
Scopes of Attributes

- 'attribute' in ServletContext can be used by all the servlets in web application.
- 'attribute' in ServletRequest of one servlet is available to next Servlet program in forward communication process.
- 'attribute' in HTTPSESSION is available from session start to end.(ex:- login,logout).

DAO(Data Access Object) Layer:

=>DAO stands for Data Access Object and which is separate layer in MVC

(Model View Controller) to hold DataBase related operations like Connection, create,Select(retrieve),insert,update and delete.



Task:- create one web application includes database and complete flow.

HttpServlet

->HttpServlet is an abstract class from 'javax.servlet.http' package. This is protocol dependent (HTTP protocol dependent).

->as we know we have Servlet,GenericServlet and now HttpServlet ..so like wise others we need to extend our servlet with HttpServlet .

->HttpServlet provides method like **doPost()** , **doGet()**..etc.

define request?

=>The query which is generated from user through WebBrowser is known as 'request'.

-> *request is categorized into two types:*

- 1.POST request
- 2.GET request

POST request:

->The request which is used to post(send) the data to the Server is known as POST request. The Post request is generated by declaring **method="POST"** on form tag.

syntax:

```
<form action="url" method="post">
```

.

.

```
</form>
```

->we use doPost() method from "javax.servlet.http.HttpServlet" interface to hold POST request.

-> Through POST request we can send any type of data (Text,Audio,Video,Image and Animation) and Unlimited data.

-> The data which is sent through the POST request is secured.

GET request:

->The request which is used to get the data from the Server is known as GET request.

The GET request is generated in the following ways:

1.By declaring method="GET" in <form> tag of HTML file.

syntax:

```
<form action="url" method="GET">
```

....

```
</form>
```

2.By using Hyper Link ex:- `<a href="index.jsp">NewUser?</a>`

3. By using url-pattern in address bar

Exp:

`http://localhost:8082/codemines/abc`

4. Submit the HTML form without declaring "method" attribute

syntax:

```
<form action="url">
```

.

.

```
</form>
```

-> we use `doGet()` method from "javax.servlet.http.HttpServlet" interface to hold GET request.

Note:

->Through GET request we can send only text data and we can send only limited data upto few kb.(Based on Browser).

-> The data which is sent through the GET request is NotSecured because which is attached to the query-string and display in the address bar.

Session Tracking /session management

->session tracking is a way to maintain state(data) of an user . in simple word process of tracking the status of user from login to logout is know as session tracking.

->following are the techniques used in session tracking.

- URL Re-writing.
- Hidden Form Fields
- Cookie (imp)
- HttpSession (imp)

->session tracking technique are used to send the information from one servlet to another servlet in the process of Tracking users.

Url Re-Writing :

=>The process of adding parameter-values to the url-pattern is known as

URL Re-Writing process.

Syn

url-pattern?para1_name=value¶2_name=value¶3_name=value&...

ex

input.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
  <form action="firstServlet" method="post">
    UserName:<input type="text" name="uname"><br>
    MailId:<input type="text" name="mid"><br>
    PhoneNo:<input type="text" name="phno"><br>
    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

```
@WebServlet("/firstservlet")
```

```
public class FirstServlet extends HttpServlet{
```

```
@Override
```

```
protected void doPost(HttpServletRequest req,
```

```
HttpServletResponse res)throws ServletException,
```

```
IOException{
```

```
PrintWriter pw = res.getWriter();
```

```
res.setContentType("text/html");
```

```
String uName = req.getParameter("uname");
```

```
String mId = req.getParameter("mid");
```

```
long phNo = Long.parseLong(req.getParameter("phno"));
```

```
pw.println
```

```
("<a href='secondservlet?uname="+uName+"&mid="+mId+"&phno="+
```

```
phNo+">ViewInfo</a>");
```

```
}
```

```
}
```

```
@WebServlet("/secondservlet")
```

```
public class SecondServlet extends HttpServlet{
```

```
@Override
```

```
protected void doGet(HttpServletRequest req,
```

```
HttpServletResponse res)throws ServletException,
```

```
IOException{
```

```
PrintWriter pw = res.getWriter();
```

```
res.setContentType("text/html");
```

```
pw.println("UserName:"+req.getParameter("uname"));
```

```
pw.println("<br>MailId:"+req.getParameter("mid"));
```

```
pw.println("<br>PhoneNo:"+req.getParameter("phno"));}}
```

Hidden Form fields:

->The process of declaring `<input type="hidden">` in form tag of HTML is known as Hidden Form Field.

->The data in Hidden form fields are not displayed to the End-Users. Using these Hidden form fields we can send information from one Servlet to another Servlet.

input.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
  <form action="firstservlet" method="post">
    UserName:<input type="text" name="uname"><br>
    MailId:<input type="text" name="mid"><br>
    PhoneNo:<input type="text" name="phno"><br>
    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

@WebServlet("/firstservlet")

public class FirstServlet extends HttpServlet{

protected void doPost(HttpServletRequest req,HttpServletResponse res)throws ServletException,

IOException{

PrintWriter pw = res.getWriter();

res.setContentType("text/html");

String uName = req.getParameter("uname");

String mId = req.getParameter("mid");

long phNo = Long.parseLong(req.getParameter("phno"));

pw.println("<form action='secondservlet' method='post'>");

pw.println

("<input type='hidden' name='uname' value='"+uName+"'>");

pw.println("<input type='hidden' name='mid' value='"+mId+"'>");

```

pw.println("<input type='hidden' name='phno' value='"+phNo+"'>");
pw.println("<input type='submit' value='ViewInfo'>");
pw.println("</form>");
}
}

@WebServlet("/secondervlet")

public class SecondServlet extends HttpServlet{

    protected void doPost(HttpServletRequest req,HttpServletResponse res)throws ServletException,
    IOException{

        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");
        pw.println("UserName:"+req.getParameter("uname"));
        pw.println("<br>MailId:"+req.getParameter("mid"));
        pw.println("<br>PhoneNo:"+req.getParameter("phno"));

    }
}

```

Cookie

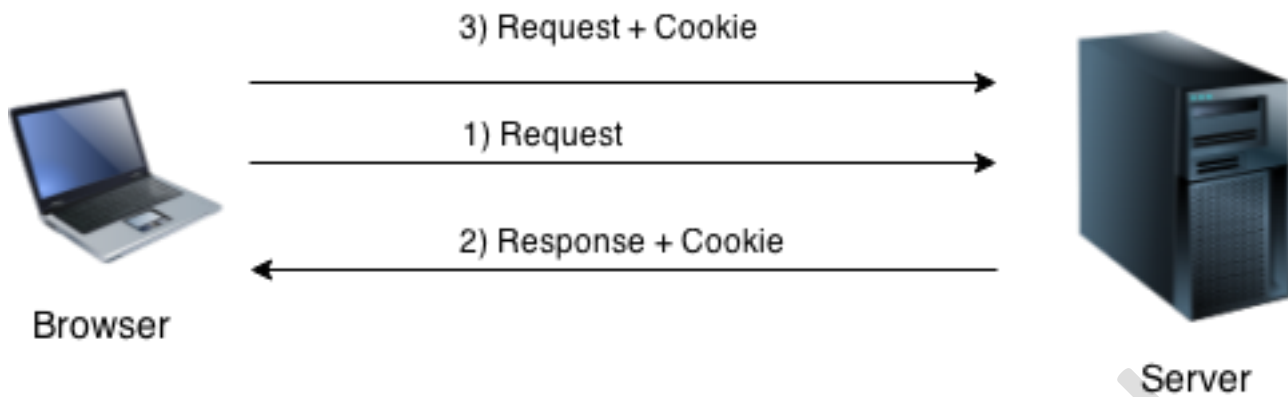
->The small piece of information available b/w multiple client request is know as 'Cookie'.

->The Cookie is Stored in WebBrowser to track the user or client.

->we use 'javax.servlet.http.Cookie' class in creating cookie.

How Cookie Works

->By default, each request is considered as a new request. In Cookies technique. We add cookie with response . So cookie get stored in the browser. After that if any request is sent by the user, Cookie is added with request by default. Thus our server program can easily recognize that the user is a old user.



The following are some important methods from Cookie class:

public void setMaxAge(int); -set maximum age of the cookie in seconds.

public int getMaxAge(); - help to get maximum age of cookie.

public void setValue(java.lang.String); -changes or set the value of cookie.

public java.lang.String getValue(); - return the value of cookie.

public java.lang.String getName(); - return the name of the cookie.

We use the following steps in cookie Session Tracking process:

step-1 : Creating cookie by creating object for 'Cookie' class as follows:

```
Cookie ck = new Cookie("name","value");
```

step-2 : Add the Cookie to the response using addCookie() method

```
res.addCookie(ck);
```

NOTE:- This addCookie() method is available from 'HttpServletResponse'.

step-3 : Get cookies from the request object using getCookies() method

```
Cookie c[] = req.getCookies();
```

NOTE:- this get_cookies() methods is available from 'HttpServletRequest'.

step-4 : Invalidating cookie

->We can invalidate cookie by setting maxAge as zero

ck.setMaxAge(0);

ex:-

```
index.jsp FirstServlet.java SecondServlet.java
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4 <html>
5 <head>
6 <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <form action="firstservlet" method="post">
11   UserName:<input type="text" name="uname"><br>
12   MailId:<input type="text" name="mid"><br>
13   PhoneNo:<input type="text" name="phno"><br>
14   <input type="submit" value="Submit">
15 </form>
16
17 </body>
18 </html>

@WebServlet("/firstservlet")
public class FirstServlet extends HttpServlet{

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        super.doGet(req, resp);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");

        Cookie ck1=new Cookie("uname",req.getParameter("uname"));
        Cookie ck2=new Cookie("mid",req.getParameter("mid"));
        Cookie ck3=new Cookie("phno",req.getParameter("phno"));

        //add these cookies to response
        res.addCookie(ck1);
        res.addCookie(ck2);
        res.addCookie(ck3);
        pw.println("<a href='secondServlet'>ViewInfo</a>");
    }

}
```

```

@WebServlet("/secondServlet")
public class SecondServlet extends HttpServlet{

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
        PrintWriter pw = res.getWriter();
        res.setContentType("text/html");

        //getting cookies in this servlet
        Cookie c[] = req.getCookies();

        for(Cookie cok:c)
        {
            pw.println("<br>" + cok.getName() + " " + cok.getValue());
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        // TODO Auto-generated method stub
        super.doPost(req, resp);
    }

}

```

search.searchpdf2.c... download setup of... YouTube

UserName:

MailId:

PhoneNo:

[ViewInfo](#)

Application									
Name	Value	D.	Path	Expi...	Size	H...	S...	S...	P...
uname	ssssssssss	I...	/coo...	Sess...	15				
phno	123456	I...	/coo...	Sess...	10				
JSESSI...	0B4969B3...	I...	/coo...	Sess...	42	✓			
mid	aaaa@d.co...	I...	/coo...	Sess...	13				

HttpSession:

->HttpSession is an interface from javax.servlet.http package and which is used to transfer the information from one Servlet to another Servlet .

->Following are some important methods of HttpSession

public abstract void **setAttribute**(java.lang.String,java.lang.Object);

public abstract java.lang.Object **getAttribute**(java.lang.String);

public abstract void **removeAttribute**(java.lang.String);

public abstract void **putValue**(java.lang.String,java.lang.Object);

public abstract java.lang.Object **getValue**(java.lang.String);

public abstract void **removeValue**(java.lang.String);

public abstract **void invalidate**();

Note:

=>we use getSession() method from 'HttpServletRequest' to create the implementation object of 'HttpSession' interface.

syntax-1:

HttpSession hs = req.getSession();

=>In this syntax it checks the session object is available or not,

=>if available access the existing session object reference.

=>if Not-Available then create new session object.

=>No parameter in this because it will create fresh session object.

syntax-2:

HttpSession hs = req.getSession(false);

=>In this syntax it checks the session object is available or not,

=>if available access the existing session object reference

=>if Not-Available then don't create new session object

=>this method is having parameter false. Means it will help to get session which is already available and if not available then it will create new one.

```
index.jsp  FirstServlet.java  SecondServlet.java

1  <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4  <html>
5  <head>
6    <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7    <title>Insert title here</title>
8  </head>
9  <body>
10   <form action="firstservlet" method="post">
11     UserName:<input type="text" name="uname"><br>
12     MailId:<input type="text" name="mid"><br>
13     PhoneNo:<input type="text" name="phno"><br>
14     <input type="submit" value="Submit">
15   </form>
16
17 </body>
18 </html>

>
4  @WebServlet("/firstservlet")
5  public class FirstServlet extends HttpServlet{
6
7      @Override
8      protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
9
10         super.doGet(req, resp);
11     }
12
13     @Override
14     protected void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
15         PrintWriter pw = res.getWriter();
16         res.setContentType("text/html");
17
18         HttpSession sess = req.getSession();//creating fresh new session
19         sess.setAttribute("uname",req.getParameter("uname"));
20         sess.setAttribute("mid",req.getParameter("mid"));
21         sess.setAttribute("phno",req.getParameter("phno"));
22
23         pw.println("Session Created...<br>");
24         pw.println("<a href='secondservlet'>ViewInfo</a>");
25     }
26 }

27
28
29 @WebServlet("/secondservlet")
30 public class SecondServlet extends HttpServlet{
31
32     @Override
33     protected void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
34         PrintWriter pw = res.getWriter();
35         res.setContentType("text/html");
36
37         HttpSession hs = req.getSession(false);//Accessing session
38         pw.println("UserName:"+hs.getAttribute("uname"));
39         pw.println("<br>MailId:"+hs.getAttribute("mid"));
40         pw.println("<br>PhoneNO:"+hs.getAttribute("phno"));
41     }
42
43     @Override
44     protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
45         // TODO Auto-generated method stub
46         super.doPost(req, resp);
47     }
48 }
49
50
51
```

Session

Session are server-side files that contains user data.

When user quits the browser or logout, the session is over.

It can hold an indefinite quantity of data.

We can keep as much data as we like within a session.

Session are more secured as compare to cookies.

Cookies

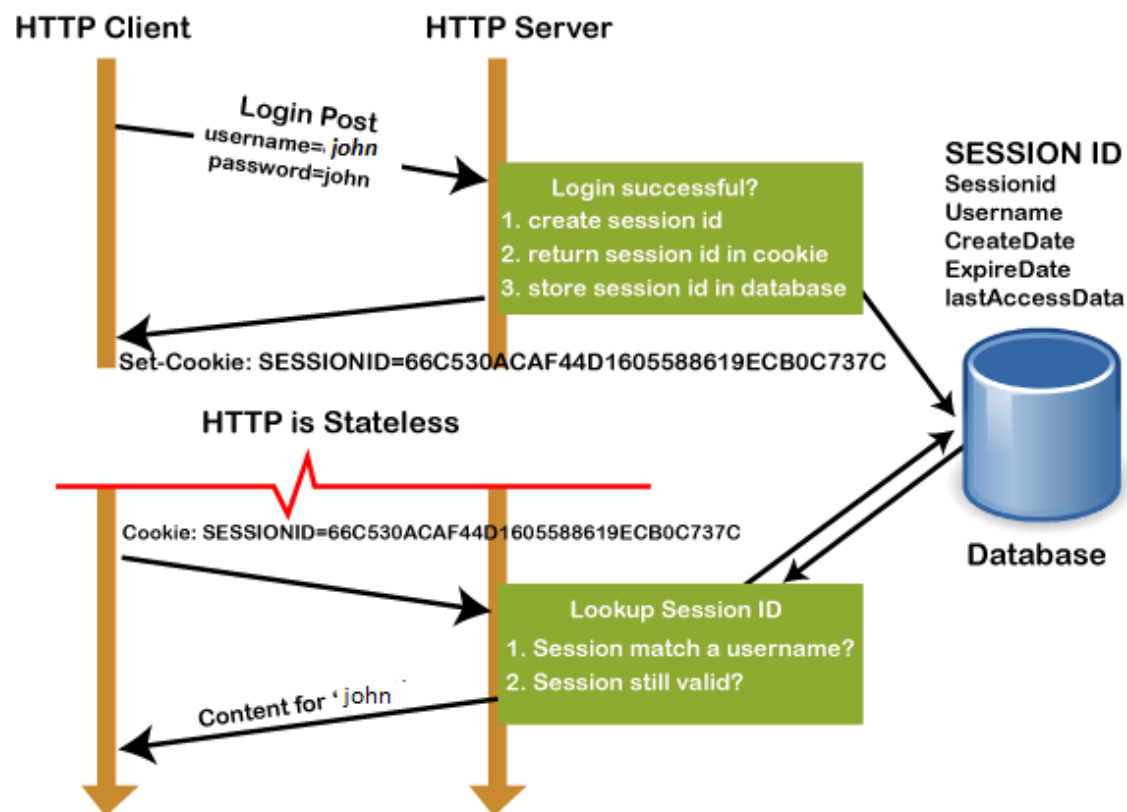
Cookies are client-side files that are stored on local computer and contain user information.

Cookies expire after the user specified lifetime.

It can store a limited amount of data.

Cookies can only store up to max 4kb data.

Cookies is not dependent on Session.



Every session will have SessionID. And Session ID is a unique number, server assigns to a specific user, during his visit(session). And defaultely, session ID is attached to a cookie and this cookie will be shared from client to server (and server to client) during its requests/responses. And server will identify session based on session id which is retrieved from cookie.

Filters in Servlet programming:

->Filter is a pre-processing component and which is executed before Servlet Program.

->Filters are used to get information from incoming requests.

->Filter are used to pre-process the request a/c to business requirement.

->Filter can also be used for encryption and decryption.

->Filters can be used to validate requests data.

=>The following are the components used in Constructing Filter Programs:

1.Filter

2.FilterChain

3.FilterConfig

1.Filter:

=>Filter is an interface from javax.servlet package and which is implemented to user defined class in the process of constructing filter program.

->The following are the Life-Cycle methods of Filter:

- Init()
- doFilter
- destroy

2.FilterChain:

->This doFilter() method of FilterChain is used to establish link b/w Filter program to Servlet program.

->FilterChain is an interface from javax.servlet package and which is instantiated automatically while executing doFilter() method from "Filter" interface.

3.FilterConfig:

->FilterConfig is an interface from javax.servlet package and which is instantiated automatically when Filter program loaded for execution.

->This FilterConfig object will hold filter name.

Note:

=>**The Filter program is mapped using Annotation or web.xml**

Case-1 : We use the following syntax in annotation

@WebFilter("/url-pattern")

Case-2 : we use the following structure of web.xml to map Filter program

<web-app>

<filter>

<filter-name> name </filter-name>

<filter-class> class </filter-class>

<init-param>

<param-name> name </param-name>

<param-value> value </param-value>

</init-param>

</filter>

<filter-mapping>

<filter-name> name </filter-name>

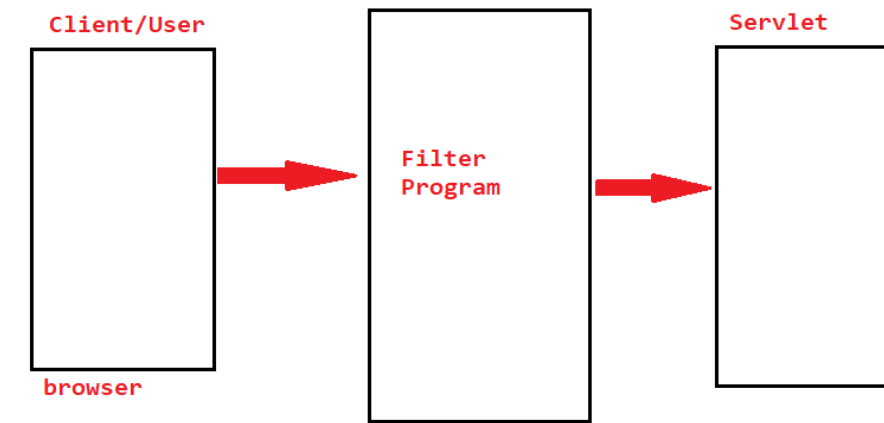
<url-pattern> /url </url-pattern>

</filter-mapping>

.

.

</web-app>



Servlet Collaboration process?

->in simple process of exchanging information b/w Servlet programs is known as Servlet Collaboration process.

->The servlet Collaboration process can be done in two ways.

- Using 'RequestDispatcher'
- Using 'sendRedirect()' method.

(a)using 'RequestDispatcher':

->'RequestDispatcher' is used to establish communication b/w Servlet

programs available in same WebApplication.

->as we already discussed This 'RequestDispatcher' provide the following two methods to perform

Communication process:

(i)forward() - forward communication process

(ii)include() - include communication process

(b)Using 'sendRedirect() method':

->using sendRedirect() method is used to establish communication b/w Servlet Program running in different web application.

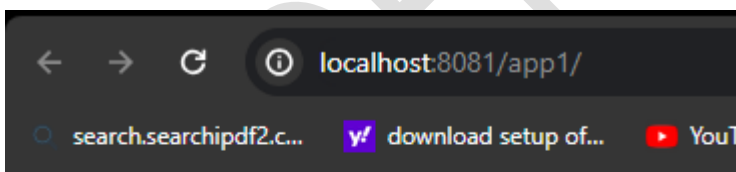
->in this web application are executed in same webserver or different webserver.

->this is available in 'HttpServletResponse'.

syntax:

```
res.sendRedirect("http://.....");
```

ex



UserName:

MailId:

```

1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4 <html>
5 <head>
6 <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <form action="first" method="post">
11   UserName:<input type="text" name="uname"><br>
12   MailId:<input type="text" name="mid"><br>
13   <input type="submit" value="Display">
14 </form>
15
16 </body>
17 </html>

```

```

1 @WebServlet("/first")
2 public class Mycontroller extends HttpServlet {
3
4     @Override
5     protected void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
6
7         String uName = req.getParameter("uname");
8         String mId = req.getParameter("mid");
9         res.sendRedirect("http://localhost:8081/app2/second?uname="+uName+"&mid="+mId);
10
11     }
12 }

```

```

1 package controller;
2
3 import java.io.IOException;
4
5
6 @WebServlet("/second")
7 public class MyServlet extends HttpServlet {
8
9     @Override
10    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException
11    {
12        PrintWriter pw = res.getWriter();
13        res.setContentType("text/html");
14
15        pw.println("===SecondServlet===");
16        pw.println("<br>UserName:" + req.getParameter("uname"));
17        pw.println("<br>MailId:" + req.getParameter("mid"));
18    }
19 }

```

Note:- server make use of browser and redirect to that URL.

Note:-When we use RequestDispatcher the same Old request is forwarded to next Servlet Program in communication process.

Note:-When we use sendRedirect() method then app1 will take help of browser and generate new request to app2.

Some important Annotations in Servlet Programming:

->we have used lots of annotation in our servlet like

- @WebServlet
- @WebFilter
- @WebListener

(i)@WebServlet

->This @WebServlet annotation will hold url-pattern and provide information to execution_control to map Servlet program for execution.

(ii)@WebFilter

->This @WebFilter annotation will hold url-pattern and provide information to execution_control to map Filter program for execution.

(iii)@WebListener

->This @WebListener annotation will provide information to execution_control to map Listener program for execution.

Servlet Life-cycle

->in real life also every one have there life cycle which means from starting stage to end stage. Similarly Servlet life-Cycle defines different state of Servlet program from Starting to Ending of Servlet Program.

following are the stages of Servlet Life-Cycle:

- Loading process
- Instantiation process
- Initialization process
- Request handling process
- Destroying process.

1.Loading Process:- process of identifying the servlet program based on URL-pattern and loading in WebContainer is know as **Loading Process**.

2.InstantiationProcess:-Once Servlet program loaded into webContainer, it is instantiated automatically .(creating object for servlet program).

->after instantiation process we can identify the following life-cycle methods.

GenericServlet:

- (i)init()
- (ii)service()
- (iii)destroy()

HttpServlet:

- (i)init()
- (ii)doPost()/doGet()
- (iii)destroy()

Filter:

- (i)init()
- (ii)doFilter()
- (iii)destroy()

3.Initialization Process:- process making the programming component ready which can be used by following methods like service(),doPost(),doGet(),doFilter().

->This initialization process is performed using init() method.

Note:

->init() method executes only once in the process of executing Life-Cycle

4.Request Handling process:

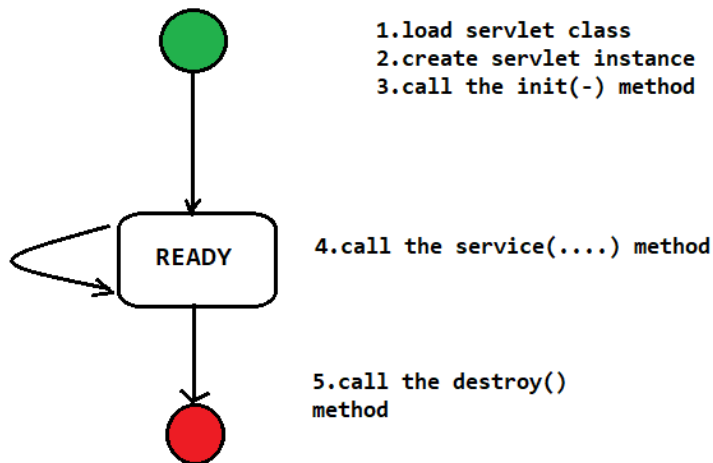
=>The process of executing service()/doPost()/doGet()/deFilter() method to handle the request is known as Request Handling process.

Note:

->service() method is executed for all the multiple requests generated from Multiple users.

5.Destroying process:

->After service() method execution,the destroy() method is executed to destroy the instances of Servlet program.

**JSP Programming:(Part-3)**

->JSP stands for 'Java Server Page' and which is response from web-application.

->JSP is tag based programming language and which is more easy when compared to Servlet Programming.

->we need to save JSP file with .jsp extension.

->in more simple words you can say JSP program are combination of both HTML code and JAVA code.

->JSP provides various tags to write java code part of JSP program.

1.Scripting tags

- Scriptlet tag
- Expression tag
- Declarative tag

2.Directive tags

- Page
- Include
- Taglib

3.Action tags

- Jsp:include
- Jsp:forward
- Jsp:param
- Jsp:useBean
- Jsp:setProperty
- Jsp:getProperty

1.Scripting tags

->it is used to write java code as part of JSP program.

->it is categorized into following tags **Scriptlet tag , Expression tag,Declarative tag.**

(a)Scriptlet tag:

->Scriptlet tag is used to write normal Java Code part of JSP programs.

syntax:

```
<% ---JavaCode--- %>
```

(b)Expression tag:

->Expression tag is used to assign the value to variable or which is used to display the data to the WebBrowser.

syntax:

`<%= expression %>`

(c)Declarative tag:

->Declarative tag is used to declare variables and methods in JSP programs.

syntax:

`<%! variables;methods %>`

2.Directive tags:

-> The tags which are used to specify the directions in translation process are known as Directive Tags.

->this tag is categorized in **page,include,taglib.**

(a)page:

->'page' directive tag specifies the translator to add the related attribute to current JSP page.

syntax:

`<%@ page attribute="value" %>`

exp:

`<%@ page import="java.util.*"%>`

List of attributes:

- 1.import
- 2.contentType
- 3.extends
- 4.info
- 5.buffer
- 6.language
- 7.isELIgnored
- 8.isThreadSafe

9.autoFlush

10.session

11.pageEncoding

12.errorPage

13.isErrorPage

(b)include:

->'include' directive tag specifies the file to be included to current JSP page.

syntax:

```
<%@ include file="file-name"%>
```

Exp:

```
<%@ include file="input.html" %>
```

(c>taglib:

->'taglib' directive tag specifies to add specified url to current JSP page and which is used part of EL(Expression Lang) and JSTL (JSP Standard Tag Lib).

syntax:

```
<%@ taglib url="urloftaglib" prefix="prefixoftaglib"%>
```

Ex:-

input.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
  <form action="Calculate.jsp" method="post">
    Enter the Value:<input type="text" name="v"><br>
    <input type="submit" value="Factorial">
  </form>
</body>
</html>
```

```

<%% page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1" errorPage="Error.jsp"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%!
int fact;
int factorial(int n)
{
    fact=1;
    for(int i=n;i>=1;i--)
    {
        fact=fact*i;
    }
    return fact;
}
%>

<%
int v = Integer.parseInt(request.getParameter("v"));
int result = factorial(v);
out.println("Factorial="+result+"<br>");
%>

<%%include file="input.html" %>
</body>

```

```

<%% page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1" isErrorPage="true"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
out.println("Enter only Integer values...<br>");
%>

<%= exception %>
<br>
<%%include file="input.html" %>
</body>
</html>

```

The following are the implicit objects generated in JSP programming:

JSP name	Servlet name
1.application	- javax.servlet.ServletContext
2.config	- javax.servlet.ServletConfig
3.request	- javax.servlet.http.HttpServletRequest
4.response	- javax.servlet.http.HttpServletResponse
5.out	- javax.servlet.jsp.JspWriter
6.session	- javax.servlet.http.HttpSession
7.exception	- java.lang.Throwable
8.page	- java.lang.Object
9.pageContext	- javax.servlet.jsp.PageContext

3.Action tags:

->Action Tags are used to include some basic actions like inserting some other page resources ,forwarding the request to another page, creating or locating the JavaBean instances and,setting and retriving the bean properties in JSP pages.

Note:

->These action tags are used in Execution process at runtime.

=>The following are some Important action tags available in JSP:

- 1.<jsp:include>
- 2.<jsp:forward>
- 3.<jsp:param>
- 4.<jsp:useBean>
- 5.<jsp:setProperty>
- 6.<jsp:getProperty>

1.<jsp:include> :

->This action tag allows to include a static or dynamic resource such as HTML or JSP specified by a URL to be included in the current JSP while processing request.

-If the resource is static then its content is included in the JSP page.

->If the resource is dynamic then its result is included in the JSP page.

attributes of include tag:

page : Takes a relative URL,which locates the resource to be included in the JSP page.

```
<jsp:include page="/Header.html"/>
```

```
<jsp:include page="<%=mypath%"/>
```

2.<jsp:forward>:

-> This action tag forwards a JSP request to another resource and which can be either static or dynamic.

->If the resource is dynamic then we can use a jsp:param tag to pass name and value of the parameter to the resource.

Exp:

```
<jsp:forward page="/Header.html"/>
```

```
<jsp:forward page="<%=mypath%"/>
```

3.<jsp:param>:

->This action tag is used to hold the parameter with value and which is to be forwarded to the next resource.

4.<jsp:useBean>:

=>This tag is used to instantiate a JavaBean, or locate an existing bean instance and assign it to a variable name(id).

syntax:

```
<jsp:useBean attributes>
```

```
<!--optional body content-->
```

```
</jsp:useBean>
```

Attributes of <jsp:useBean> tag:

(a)id

(b)scope

(c)class

(d)beanName

(e)type

(a)id:

->which represents the variable name assigned to id attribute of <jsp:useBean> tag and which holds the reference of JavaBean instance.

(b)scope:

=>which specifies the scope in which the bean instance has to be created or located.

scope can be the following:

- (i)page scope : within the JSP page,until the page sends response.
- (ii)request scope : JSP page processing the same request until a JSP sends response.
- (iii)session scope - Used with in the Session.
- (iv)application scope - Used within entire web application.

(e)type :

->The "type" attribute takes a qualified className or interfaceName, which can be the classname given in the class or beanName attribute or its super type.

5.<jsp:setProperty>:

->This action tag sets the value of a property in a bean, using the bean's setter methods.

Types of attributes:

- (a)name
- (b)property
- (c)value
- (d)param

(a)name:

->The name attribute takes the name of already existing bean as a reference variable to invoke the setter method.

(b)property:

->which specifies the property name that has to be set,and specifies the setter method that has to be invoked.

(c)value:

->The value attribute takes the value that has to be set to the specified bean property.

(d)param:

->which specify the name of the request parameter whose value to be assigned to bean property.

6.<jsp:getProperty>:

->This action tag gets the value of a property in a bean by using the bean's getter method and writes the value to the current JspWriter.

Types of attributes:

(a)name

(b)property

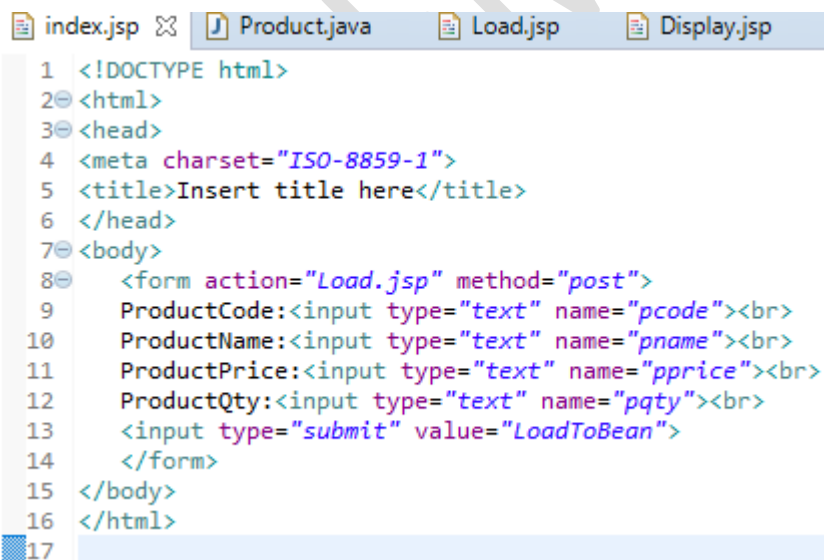
(a)name:

->The name attribute takes the reference variable name on which we want to invoke the getter method.

(b)property:

->which gets the value of a bean property and invokes the getter method of the bean property.

Demonstrating <jsp:useBean>,<jsp:setProperty> and<jsp:getProperty>



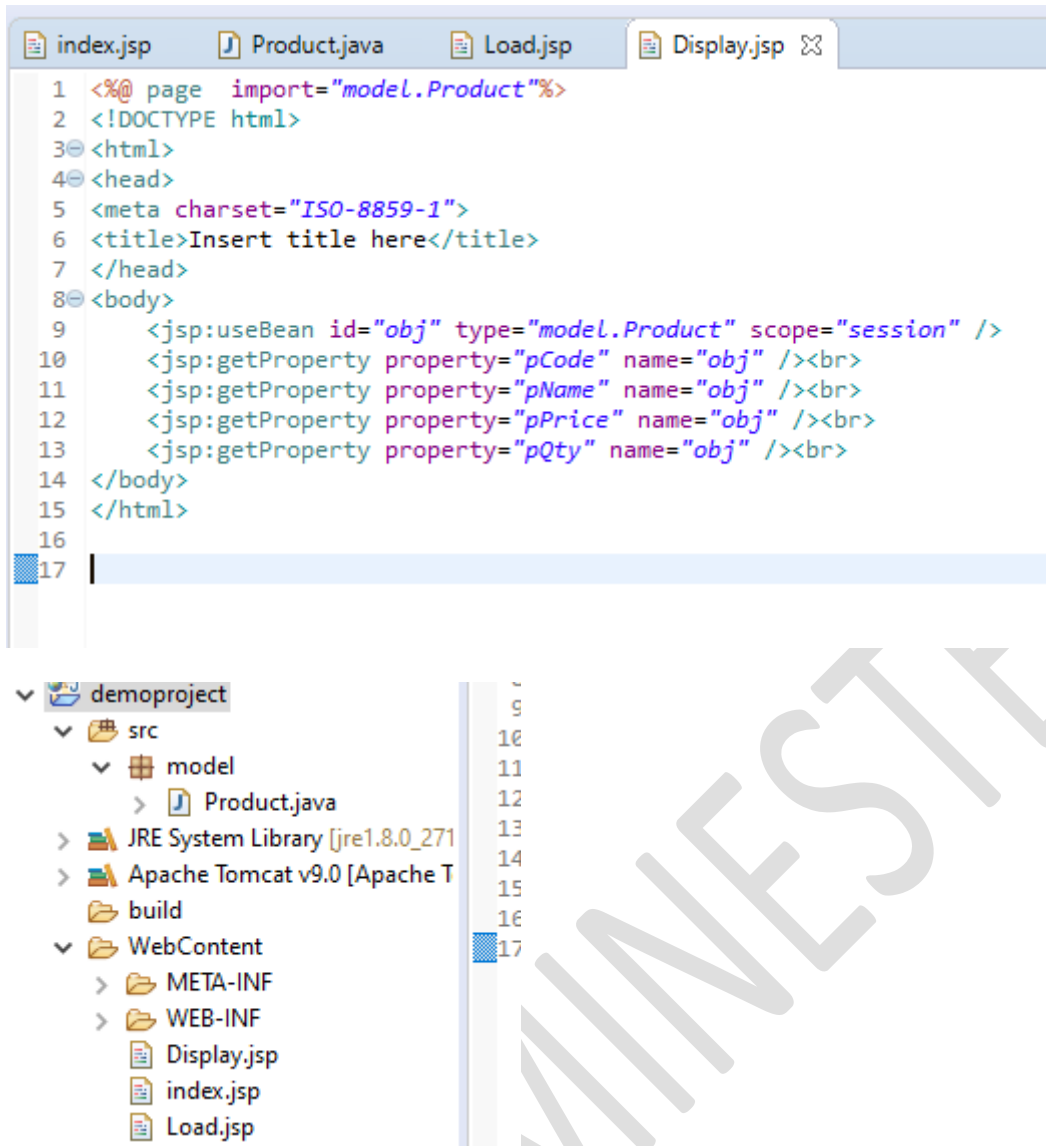
```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="ISO-8859-1">
5 <title>Insert title here</title>
6 </head>
7 <body>
8 <form action="Load.jsp" method="post">
9   ProductCode:<input type="text" name="pcode"><br>
10  ProductName:<input type="text" name="pname"><br>
11  ProductPrice:<input type="text" name="pprice"><br>
12  ProductQty:<input type="text" name="pqty"><br>
13  <input type="submit" value="LoadToBean">
14 </form>
15 </body>
16 </html>
17
```

index.jsp Product.java Load.jsp Display.jsp

```
1 package model;
2
3 public class Product {
4
5     private int pCode;
6     private String pName;
7     private int pPrice;
8     private int pQty;
9     public int getpCode() {
10         return pCode;
11     }
12     public void setpCode(int pCode) {
13         this.pCode = pCode;
14     }
15     public String getpName() {
16         return pName;
17     }
18     public void setpName(String pName) {
19         this.pName = pName;
20     }
21     public int getpPrice() {
22         return pPrice;
23     }
24     public void setpPrice(int pPrice) {
25         this.pPrice = pPrice;
26     }
27     public int getpQty() {
28         return pQty;
29     }
30     public void setpQty(int pQty) {
31         this.pQty = pQty;
32     }
33
34
35
36 }
37
```

index.jsp Product.java Load.jsp Display.jsp

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1" import="model.Product"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6   <meta charset="ISO-8859-1">
7   <title>Insert title here</title>
8 </head>
9 <body>
10   <jsp:useBean id="obj" class="model.Product" scope="session" />
11   <jsp:setProperty property="pCode" param="pcode" name="obj" />
12   <jsp:setProperty property="pName" param="pname" name="obj" />
13   <jsp:setProperty property="pPrice" param="pprice" name="obj" />
14   <jsp:setProperty property="pQty" param="pqty" name="obj" />
15   <%
16       out.println("Data loaded onto Bean object...<br>");
17   %>
18   <a href="Display.jsp">ViewData</a>
19 </body>
20 </html>
21
```



Web Architecture Models:(Web Application Architectures)

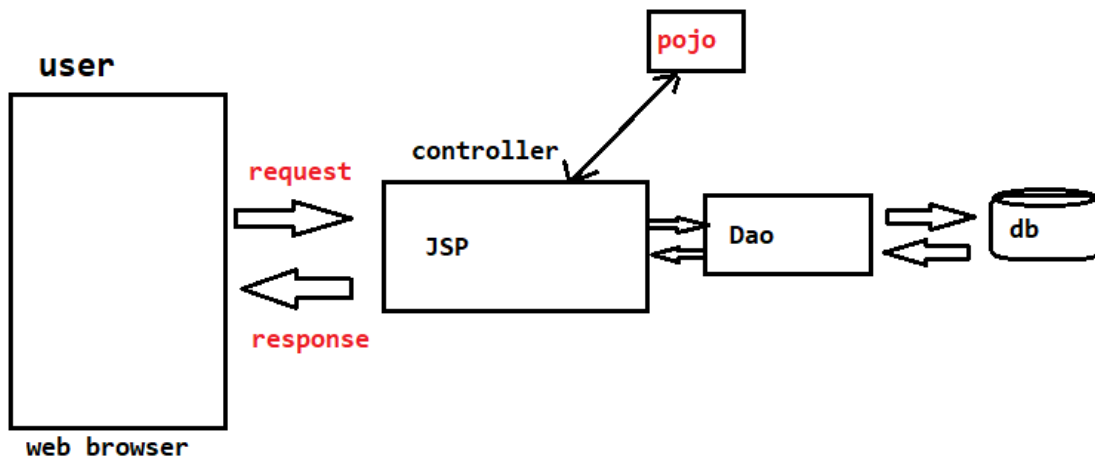
Two types of development models are used in Java for Web applications, and these models are classified based on the different approaches used to develop Web applications.

These models are:

1. Model-1 Architecture
2. Model-2 Architecture(MVC)

1. Model-1 Architecture:

The Model-1 architecture was the first development model used to develop Web applications and this model uses JSP to design applications and, which is responsible for all the activities and functionalities provided by the application.



Limitations of the Model-1 Architecture:

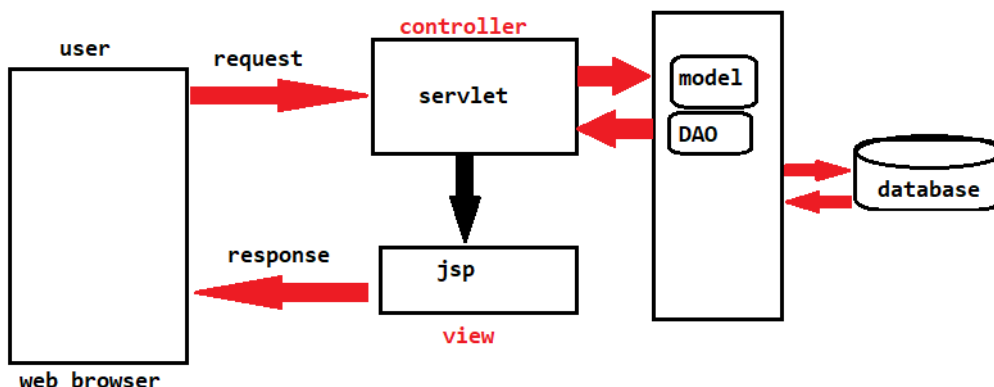
(i). Applications are inflexible and difficult to maintain. A single change in one page may cause changes in other pages, leading to unpredictable results.

(ii). Involves the developer at both the page development and the business logic implementation stages.

(iii). Increases the complexity of a program with the increase in the size of the JSP page.

2. Model-2 Architecture (MVC model)

->The drawbacks in the Model-1 architecture led to the introduction of a new model called Model-2. This Model-2 architecture is based on the MVC design model.



->MVC Stands for Model View Controller.

Model: Represents enterprise data .

View: Shows the contents of a Model.The View component accesses enterprise data through the Model component and specifies how that data should be presented and this View Component is designed by JSP.

Controller: Receives HTTP requests. The Controller component receives requests from a client, determines the business logic to be performed, and delegates the responsibility for producing the next phase of the user interface to an appropriate view component.

Advantages of Model-2 Architecture:

->Offers great flexibility to the presentation logic, which can be modified without effecting the business logic.

Expression Language(EL) in JSP:

->This EL simplifies the accessibility of data stored in java Bean component and other objects like request,session,application,etc..

Note: It is newly added feature in JSP technology.

syntax of EL:

\$(expression)

The following are the implicit objects of EL:

pageScope : It maps the given attribute name with the value,set in the page scope

requestScope : It maps the given attribute name with the value set in the request scope.

sessionScope : It maps the given attribute name with the value set in the session scope.

applicationScope : It maps the given attribute name with the value set in the application scope.

param : It maps the request parameter to the single value

paramValues : It maps the request parameters to the array of values.

header : It maps the request header name to the single value.

headerValues : It maps the request header names to the array of values.

cookie :It maps the cookie name to the cookie value.

initParam : It maps the Initialization parameter .

pageContext : It provides access to many objects request,session,...

Note:

->we use "taglib" directive tag to declare JSTL Tags.

->To Execute JSTL Tags we use the following steps:

(i)Download the following Jar files to execute JSTL tags

javax.servlet.jsp.jstl-1.2.1.jar

javax.servlet.jsp.jstl-api-1.2.1.jar

(ii)These jars must be available within 'lib' folder of 'WEB-INF' in deployment directory

(1)Core Tags:

->These JSTL core tags provides variable support,URL management, flow control etc.
(Basic code writing)

syntax:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

The following are the List of JSTL Core Tags:

c:out ->It displays the result of an expression, similar to

<%=...%> tag.

c:import->It Retrives relative or an absolute URL.

c:set-> It sets the value to variable.

c:remove->It is used for removing the variable .

c:catch-> It is used for Catching any Throwable exception that occurs in the body.

c:if-> It is an conditional tag

c:choose, c:when, c:otherwise->

It is the simple conditional tag that includes its body content

if the evaluated condition is true.

***imp**

c:forEach-> It is the basic iteration tag.

c:redirect-> It redirects the browser to a new URL and supports the context-relative URLs.

```
input.jsp  display.jsp
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="ISO-8859-1">
5 <title>Insert title here</title>
6 </head>
7 <body>
8 <form action="display.jsp" method="post">
9   val1:<input type="text" name="op1"><br>
10  val2:<input type="text" name="op2"><br>
11  <input type="submit" value="Add" name="submit">
12  <input type="submit" value="Sub" name="submit">
13 </form>
14 </body>
15 </html>
16
```

```
input.jsp  display.jsp
1
2 <%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
3 <html>
4 <head>
5 <title>EL App1</title>
6 </head>
7 <body>
8 <c:choose>
9 <c:when test="${param.submit eq 'Add'}">
10 Sum of <c:out value="${param.op1}" />
11 and
12 <c:out value="${param.op2}" /> is
13 <c:out value="${param.op1 + param.op2}" />
14 </c:when>
15
16 <c:otherwise>
17 <c:out value="${param.op1}" /> and
18 <c:out value="${param.op2}" /> is
19 <c:out value="${param.op1 - param.op2}" />
20 </c:otherwise>
21 </c:choose>
22
23 <jsp:include page="input.jsp" />
24 </body>
25 </html>
26
```

JSP LifeCycle:

->JSP LifeCycle demonstrates diff states of JSP program from Starting of the program to the ending of the program.

->The following are the states of JSP program in LifeCycle:

- (1)Translation process
- (2)Compilation process
- (3>Loading process
- (4)Instantiation process
- (5)Initialization process
- (6)Request Handling process
- (7)Destroying process

(1)Translation process:

->The process of separating JavaCode from JSP program is known as Translation process.

->After translation process SourceCode is generated.

(2)Compilation process:

=>The process of compiling the SourceCode and generating the ByteCode is known as Compilation process.

(3>Loading process:

->The process of loading the JSP program for execution is known as Loading process.

(4)Instantiation process:

=>The process of creating object for JSP program is known as Instantiation process.

=>After Instantiation process we can find the following implicit

lifeCycle methods:

(i)_jspInit()

(ii)_jspService()

(iii)_jspDestroy()

(5)Initialization process:

=>The process of making the programming components ready for service is known as Initialization process.

(6)Request Handling process:

=>The process of handling the request and giving the response is known as Request Handling process.

(7)Destroying process:

=>The process of destroying the instances from the WebContainer is known as Destroying process.

Eclipse configuration (in recent version)

If you are using recent version eclipse then you won't get webcontent folder you will get webapp so you need to put all your html,jsp ..etc in webapp only. And while adding servlet-api.jar please take care of classpath option.